

Node.js

Node.js

Check sub-tabs for complete QnA

Node.js Fundamentals

Node.js Fundamentals

Q1. What is Node.js?

Node.js is a **JavaScript runtime environment** that allows JavaScript to run outside the browser, mainly for backend/server-side development. It is built on **Chrome's V8 JavaScript engine** and uses an event-driven, non-blocking I/O model.

Key Points:

1. Runs JavaScript on the server.
2. Uses **non-blocking, asynchronous I/O**.
3. Built on the **V8 engine**.
4. Commonly used for APIs, real-time apps, and web servers.

Example:

```
const http = require("http");

http.createServer((req, res) => {
  res.end("Hello from Node.js");
}).listen(3000);
```

Here, Node.js creates a simple HTTP server that listens on port **3000**.

Q2. Is Node.js a Programming Language, Framework, or Runtime?

Node.js is a runtime environment, not a programming language or framework. It allows JavaScript to execute outside the browser using Google's **V8 JavaScript engine**.

1. **JavaScript** → Programming language.
2. **Node.js** → Runtime environment for executing JavaScript.
3. **Express.js** → Backend framework built on Node.js.

Example:

```
console.log("Running JavaScript with Node.js");
```

Here, **JavaScript is the language**, while **Node.js provides the environment to execute it**.

Q3. Why is Node.js called a JavaScript Runtime?

Node.js is called a **JavaScript runtime** because it provides the environment and necessary APIs to **execute JavaScript outside the browser**, such as on a server.

1. **V8 Engine** → Executes JavaScript code.
2. **Node.js APIs** → Provide access to files, networking, HTTP, etc.
3. **Event Loop** → Handles asynchronous and non-blocking operations.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});
```

Here, Node.js provides the **fs** API, allowing JavaScript to interact with the file system. This is something browser JavaScript normally cannot do.

Q4. What is V8, and what role does it play in Node.js?

V8 is Google's open-source JavaScript engine, written in C++, originally developed for Chrome. Node.js uses V8 to **compile and execute JavaScript code**.

Role of V8 in Node.js:

1. Executes JavaScript code.
2. Compiles JavaScript into optimized machine code.
3. Manages memory and garbage collection.
4. Provides the JavaScript execution engine; Node.js adds APIs like **fs**, **http**, and **path**.

Flow:

```
JavaScript Code → Node.js → V8 → Machine Code
                ↓
                Node.js APIs
```

Example:

```
console.log("Hello Node.js");
```

Q5. What is the Node.js Event Loop?

The **Event Loop** is the mechanism that allows Node.js to handle **asynchronous, non-blocking operations** while using a single JavaScript thread. It continuously checks for completed async tasks and executes their callbacks.

How it works:

1. Executes synchronous code on the **call stack**.
2. Sends asynchronous operations to the OS/Node.js APIs.
3. When completed, their callbacks are queued.
4. The Event Loop moves callbacks to the call stack when it is free.

Example:

```
console.log("Start");  
  
setTimeout(() => console.log("Async"), 0);  
  
console.log("End");
```

Output:

```
Start  
End  
Async
```

The `setTimeout` callback waits until the current synchronous code finishes, demonstrating **non-blocking execution**.

Q6. Why does Node.js use an Event Loop?

Node.js uses the **Event Loop** to handle many concurrent I/O operations efficiently without creating a separate thread for each request. This makes Node.js lightweight and well-suited for **I/O-heavy applications**.

1. **Non-blocking I/O** → Doesn't wait for slow operations like DB or file access.
2. **High concurrency** → Can handle many connections efficiently.
3. **Better resource usage** → Avoids creating a thread for every request.

Example:

```
fs.readFile("data.txt", () => {
  console.log("File read complete");
});

console.log("Continue working...");
```

Node.js continues executing other code while the file is being read, and the Event Loop handles the callback when the operation completes.

Q7. Is Node.js single-threaded? What does that actually mean?

Yes, JavaScript execution in Node.js is single-threaded, meaning one main thread executes JavaScript code using the Event Loop. However, Node.js can use **multiple background threads** through the OS and **libuv** for certain asynchronous operations.

What it means:

1. One main thread executes JavaScript.
2. Event Loop handles asynchronous callbacks.
3. I/O operations can be handled by the OS or libuv's thread pool.
4. CPU-heavy JavaScript can block the main thread.

Key point:

Node.js is **single-threaded for JavaScript execution**, not necessarily single-threaded internally.

Example:

```
console.log("Start");

setTimeout(() => console.log("Async"), 1000);

console.log("End");
```

```
Start
End
Async
```

Q8. What is the difference between Synchronous and Asynchronous Execution in Node.js?

Synchronous execution blocks the program until the current operation finishes, while **asynchronous execution** allows Node.js to continue executing other code and handle the result later.

Feature	Synchronous	Asynchronous
Execution	Sequential	Non-blocking
Blocks thread	Yes	Usually no
Performance	Can be slower for I/O	Better for I/O-heavy tasks
Example	<code>readFileSync()</code>	<code>readFile()</code>

Synchronous Example:

```
const fs = require("fs");

const data = fs.readFileSync("data.txt");
console.log(data.toString());

console.log("Done");
```

Asynchronous Example:

```
fs.readFile("data.txt", (err, data) => {
  console.log(data.toString());
});

console.log("Done");
```

Output:

```
Done
[file content]
```

Key point: Asynchronous execution lets Node.js handle other work while waiting for I/O operations to complete.

Q9. What does Non-Blocking I/O mean in Node.js?

Non-blocking I/O means Node.js does not wait for an I/O operation like file reading, database queries, or network requests to finish before executing other code.

How it works:

1. Node.js starts the I/O operation.
2. It continues executing other code.
3. When the operation finishes, a callback/Promise is processed through the Event Loop.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log("File:", data);
});

console.log("Continue executing...");
```

Output:

```
Continue executing...
File: Hello
```

Key point: Non-blocking I/O allows Node.js to handle **many concurrent I/O operations efficiently without blocking the main JavaScript thread**.

Q10. What happens when Node.js executes a JavaScript file?

When Node.js runs a JavaScript file, it loads the file, passes the code to the **V8 engine for execution**, and initializes the Node.js runtime environment.

Execution Flow:

1. Node.js starts and initializes the runtime.
2. The JavaScript file is loaded and wrapped in a module.
3. **V8** parses and executes the JavaScript.
4. Node.js APIs handle operations like **fs**, **http**, etc.
5. The **Event Loop** continues handling asynchronous tasks.
6. Node.js exits when there is no more work to process.

Example:

```
console.log("Hello");

setTimeout(() => {
  console.log("Done");
}, 1000);
```

Flow:

```
Node.js
  ↓
Load JS File
  ↓
V8 Executes Code
  ↓
Async Tasks → Event Loop
  ↓
No Pending Work → Exit
```

Q11. What is the Call Stack in Node.js?

The **Call Stack** is a data structure used by the JavaScript engine to keep track of **function execution**. It follows **LIFO (Last In, First Out)** order.

How it works:

1. A function call is pushed onto the stack.
2. Its code executes.
3. When it finishes, it is removed from the stack.
4. Node.js executes JavaScript code on the main thread's call stack.

Example:

```
function first() {
  second();
}
function second() {
  console.log("Hello");
}
first();
```

Stack flow:

```
first()
  ↓
second()
  ↓
console.log()
```

After each function finishes, it is **popped from the stack**.

Key point: The Call Stack handles **synchronous JavaScript execution**, while the Event Loop helps manage asynchronous operations.

Q12. What is the Event Queue?

The **Event Queue** is a queue where callbacks from completed asynchronous operations wait until the **Call Stack is free**. The Event Loop moves eligible callbacks from the queue to the Call Stack for execution.

How it works:

1. Async operation starts.
2. Operation completes.
3. Its callback is placed in an appropriate queue.
4. Event Loop moves it to the Call Stack when the stack is empty.

Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 0);

console.log("End");
```

Output:

```
Start
End
Timer
```

Key point: Node.js has different queues/phases for different kinds of callbacks;
Promise callbacks use the microtask queue, which is processed before timer callbacks.

Q13. How does the Call Stack interact with the Event Loop?

The **Call Stack** executes JavaScript code, while the **Event Loop** continuously checks whether the stack is free and moves ready asynchronous callbacks to it.

Flow:

1. Synchronous code enters the **Call Stack**.
2. Async operations are handled by Node.js/OS/libuv.
3. Completed callbacks are placed in queues.
4. Event Loop checks the Call Stack.
5. When the stack is free, eligible callbacks are pushed onto it.

Example:

```
console.log("A");  
  
setTimeout(() => console.log("B"), 0);  
  
console.log("C");
```

Execution:

```
Call Stack → A → C → empty  
           ↑  
       Event Loop  
           ↑  
Timer callback → B
```

Output:

```
A  
C  
B
```

Key point: The Event Loop **doesn't execute callbacks itself**; it coordinates when callbacks can be moved to the Call Stack.

Q14. What are Callbacks in Node.js?

A **callback** is a function passed as an argument to another function and executed later, usually after an **asynchronous operation** completes.

1. Handle results of asynchronous operations.
2. Avoid blocking the main thread.
3. Execute code when an operation finishes.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});
```

Here, `(err, data) => { ... }` is the **callback**. Node.js executes it after the file-reading operation completes.

Key point: Callbacks are one of the traditional ways Node.js handles asynchronous operations; modern code often uses **Promises** and **async/await** instead.

Q15. What are Promises, and why are they used?

A **Promise** is an object that represents the eventual result of an asynchronous operation. It can be **pending**, **fulfilled**, or **rejected**.

1. Handle asynchronous operations more cleanly.
2. Avoid deeply nested callbacks (**callback hell**).
3. Handle success and errors using `.then()` and `.catch()`.
4. Work naturally with **async/await**.

Example:

```
const getData = () => {
  return Promise.resolve("Data received");
};

getData()
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Promise states:

```
Pending → Fulfilled  
      ↘ Rejected
```

Key point: Promises make asynchronous code **easier to read, chain, and handle errors in**.

Q16. How does **async/await** work?

async/await is syntax built on top of **Promises** that makes asynchronous code look and behave more like synchronous code. It improves readability without blocking the Node.js main thread.

1. **async** makes a function return a **Promise**.
2. **await** pauses execution of that async function until the Promise settles.
3. It does **not block the Node.js Event Loop**.
4. **try/catch** can be used to handle errors.

Example:

```
async function getData() {  
  try {  
    const data = await fetchData();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Flow:

```
async function  
  ↓  
  await Promise  
  ↓  
Event Loop handles other work  
  ↓  
Promise settles  
  ↓  
Function continues
```

Q17. What is the difference between `require` and `import`?

`require` is the **CommonJS** module syntax, while `import` is the **ES Module (ESM)** syntax. Both are used to load and share modules in Node.js.

Feature	<code>require</code>	<code>import</code>
Module system	CommonJS	ES Modules
Syntax	<code>require()</code>	<code>import / export</code>
Loading	Traditionally synchronous	Supports static/module-based loading
Dynamic loading	<code>require()</code> can be called dynamically	Use <code>import()</code>
Common usage	Older Node.js projects	Modern JavaScript/Node.js

`require` Example:

```
const fs = require("fs");
```

`import` Example:

```
import fs from "fs";
```

Export:

```
// CommonJS
module.exports = add;

// ES Module
export default add;
```

Key point: Modern Node.js supports **both**, but `import/export` is the standard JavaScript module system, while `require/module.exports` belongs to CommonJS.

Q18. Differentiate between CommonJS vs ES Modules?

CommonJS (CJS) and **ES Modules (ESM)** are two module systems used in Node.js to organize and share JavaScript code between files.

Feature	CommonJS	ES Modules
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Standard	Node.js module system	JavaScript standard
File type	<code>.js</code> by default in CommonJS projects	<code>.mjs</code> or <code>.js</code> with <code>"type": "module"</code>
Loading	Traditionally synchronous	Static module structure
Dynamic loading	<code>require()</code>	<code>import()</code>
Example	<code>const x = require("./x")</code>	<code>import x from "./x.js"</code>

CommonJS:

```
// math.js
module.exports = (a, b) => a + b;

// app.js
const add = require("./math");
```

ES Modules:

```
// math.js
export const add = (a, b) => a + b;

// app.js
import { add } from "./math.js";
```

Key point: **CommonJS** is the traditional Node.js module system, while **ES Modules** are the standardized modern JavaScript module system.

Q19. What is npm, and what is `package.json`?

npm (Node Package Manager) is the default package manager for Node.js. It is used to **install, manage, update, and publish JavaScript/Node.js packages**.

`package.json` is the project's **configuration and metadata file** that defines dependencies, scripts, project information, and other settings.

Role of npm:

1. Install packages → `npm install express`
2. Manage dependencies and versions.
3. Run project scripts → `npm start`
4. Publish packages to the npm registry.

Role of `package.json`:

1. Stores project metadata.
2. Defines `dependencies` and `devDependencies`.
3. Defines commands under `scripts`.

Example:

```
{
  "name": "my-app",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Key point:

npm = package manager/tool

`package.json` = project configuration file

Q20. What is the difference between **dependencies** and **devDependencies**?

dependencies are packages required for the application to **run in production**, while **devDependencies** are packages needed only during **development, testing, or building**.

Feature	dependencies	devDependencies
Purpose	Run the application	Develop/build/test the application
Production	Required	Usually not required
Install command	<code>npm install package</code>	<code>npm install -D package</code>
Examples	Express, Mongoose	Jest, ESLint, Nodemon

Example:

```
{
  "dependencies": {
    "express": "^5.1.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

Here, **Express** is needed to run the server, while **Nodemon** is mainly used during development.

Key point:

dependencies → needed by the application

devDependencies → needed to develop the application

Q21. What is `package-lock.json`, and why is it important?

`package-lock.json` records the **exact versions of installed dependencies and their sub-dependencies**. It ensures that the same dependency tree can be installed consistently across different machines and environments.

Why is it important?

1. **Exact versions** → Locks dependency versions.
2. **Reproducible installs** → Same versions for developers, CI/CD, and production.
3. **Dependency tree** → Records nested dependencies and resolved packages.
4. **Faster, reliable installs** → Helps npm reproduce the known dependency tree.

Example:

```
npm install
```

npm uses `package-lock.json` to install the locked dependency versions rather than resolving everything from scratch.

Difference:

File	Purpose
<code>package.json</code>	Defines required dependencies and version ranges
<code>package-lock.json</code>	Records the exact dependency versions installed

Key point:

`package.json` says "what we need", while `package-lock.json` says "exactly what was installed."

Q22. What is the purpose of Environment Variables in Node.js?

Environment variables store configuration values outside the source code, such as API keys, database URLs, ports, and environment-specific settings.

Why use them?

1. **Security** → Keep secret data out of source code.
2. **Configuration** → Change configurations/settings without modifying code.
3. **Environment separation** → Use different values for development, testing, and production.

Example:

`app.js`:

```
const port = process.env.PORT || 3000;
const dbUrl = process.env.DATABASE_URL;

console.log(`Server running on ${port}`);
```

`.env`:

```
PORT=5000
DATABASE_URL=mongodb://localhost/mydb
```

With a package like `dotenv`, these values can be loaded into `process.env`.

Key point: Environment variables allow you to **separate configuration and secrets from application code**.

Q23. What are Node.js Core Modules? Give some examples.

Node.js core modules are built-in modules provided by Node.js, so they can be used **without installing external packages**. They provide functionality for files, networking, operating system operations, paths, etc.

Examples:

1. `fs` → File system operations.
2. `http` → Create HTTP servers and clients.
3. `path` → Work with file and directory paths.

4. **os** → Get operating system information.
5. **events** → Create and handle events.
6. **crypto** → Cryptographic operations.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});
```

Key point: Core modules are **built into Node.js**, so you don't need **npm install** to use them.

Q24. What is the Node.js Runtime?

The **Node.js runtime** is the environment that allows JavaScript to execute **outside the browser**. It combines the **V8 JavaScript engine**, Node.js APIs, and the **Event Loop** to run server-side applications.

Main Components:

1. **V8 Engine** → Executes JavaScript.
2. **Node.js APIs** → Provide access to files, HTTP, OS, etc.
3. **libuv** → Handles asynchronous I/O and the Event Loop.
4. **Event Loop** → Coordinates non-blocking operations.

Flow:

```
JavaScript
  ↓
Node.js Runtime
├── V8 → Executes JS
├── Node APIs → fs, http, crypto...
└── libuv → Async I/O + Event Loop
```

Key point: **Node.js runtime = V8 + Node.js APIs + libuv + Event Loop**, providing everything needed to execute JavaScript on the server.

Q25. What is the difference between Node.js Runtime and Browser Runtime?

Both runtimes execute JavaScript using a JavaScript engine, but they provide **different environments and APIs**. Node.js is designed for server-side applications, while browsers are designed to run JavaScript inside web pages.

Feature	Node.js Runtime	Browser Runtime
Primary purpose	Server-side applications	Web applications
JavaScript engine	V8 (Node.js uses V8)	V8, SpiderMonkey, JavaScriptCore, etc.
DOM	Not available by default	Available
<code>window</code>	Not Available	Available
File system	Via <code>fs</code>	Direct access restricted
HTTP/server APIs	Built-in	Uses browser APIs like <code>fetch()</code>
Module system	CommonJS + ES Modules	ES Modules
Global object	<code>global</code> / <code>globalThis</code>	<code>window</code> / <code>globalThis</code>

Example:

```
// Node.js
const fs = require("fs");
fs.readFileSync("data.txt");

// Browser
document.querySelector("#title");
```

Key point:

Browser runtime → Web APIs + DOM

Node.js runtime → Server/OS APIs + Event Loop

Q26. What is the difference between Concurrency and Parallelism in Node.js?

Concurrency means handling multiple tasks by **making progress on them during overlapping time periods**, while **parallelism** means actually executing multiple tasks **at the same time**, usually on multiple CPU cores.

Feature	Concurrency	Parallelism
Meaning	Multiple tasks are in progress	Multiple tasks execute simultaneously
Node.js	Event Loop + async I/O	Worker Threads / multiple processes
CPU cores	Can work with one core	Typically uses multiple cores
Best for	I/O-heavy tasks	CPU-heavy tasks
Example	Multiple HTTP requests	CPU calculations on Worker Threads

Concurrency example:

```
fetchData1();  
fetchData2();  
fetchData3();
```

Node.js can start all three I/O operations without waiting for each one to finish.

Parallelism example:

```
const { Worker } = require("worker_threads");  
new Worker("./worker.js");
```

Worker Threads can execute CPU-intensive JavaScript work on separate threads.

Key point:

Concurrency = dealing with multiple tasks at once

Parallelism = doing multiple tasks at the same time.

Runtime & Architecture

Runtime & Architecture

Q27. What is the role of libuv in Node.js?

libuv is a C library used by Node.js to provide **asynchronous, non-blocking I/O**. It powers the **Event Loop** and manages operations such as file system, networking, and a thread pool for certain tasks.

Role of libuv:

1. **Event Loop** → Manages asynchronous callbacks.
2. **Async I/O** → Handles non-blocking file and network operations.
3. **Thread Pool** → Handles certain operations that cannot be performed efficiently asynchronously by the OS.
4. **Cross-platform support** → Provides consistent async I/O behavior across operating systems.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});

console.log("Continue...");
```

Here, Node.js delegates the file operation to its underlying I/O mechanisms, with **libuv coordinating the asynchronous work and callback execution**.

Key point:

V8 executes JavaScript, while libuv powers Node.js's asynchronous I/O and Event Loop.

Q28. What is the thread pool in Node.js?

The **Thread Pool** is a set of background threads managed by **libuv** that handles certain operations that could otherwise block the main JavaScript thread.

Role of Thread Pool:

1. Handles certain **file system operations**.
2. Handles **DNS lookups** and some other blocking tasks.
3. Performs **cryptographic operations** such as password hashing.
4. Keeps the main Event Loop available for other work.

Example:

```
const crypto = require("crypto");

crypto.pbkdf2("password", "salt", 100000, 64, "sha512", () => {
  console.log("Hash completed");
});

console.log("Continue...");
```

The expensive hashing operation can run in the **libuv thread pool**, allowing the main JavaScript thread to continue executing.

Key point:

Event Loop → coordinates asynchronous work

Thread Pool → executes certain expensive/blocking operations in background threads

Default: libuv's thread pool has **4 worker threads**, configurable using `UV_THREADPOOL_SIZE`.

Q29. Which operations in Node.js can use the libuv thread pool?

The **libuv thread pool** is used for certain operations that may block or don't have efficient non-blocking OS APIs. The exact behavior can depend on the platform.

Common operations:

1. **File System (fs)** → File operations such as `readFile()`, `writeFile()`.
2. **DNS** → `dns.lookup()` can use the thread pool.
3. **Crypto** → Operations such as `pbkdf2()`, `scrypt()`, and some key-generation functions.
4. **Compression (zlib)** → Certain asynchronous compression/decompression operations.

Example:

```
const crypto = require("crypto");
```

```
crypto.pbkdf2(  
  "password",  
  "salt",  
  100000,  
  64,  
  "sha512",  
  () => console.log("Done")  
);
```

```
console.log("Main thread continues...");
```

The hashing work can be handled by a **libuv worker thread**, while the main JavaScript thread continues executing.

Key point:

libuv thread pool → mainly **FS, DNS, crypto, and certain zlib operations**.

Default thread pool size: 4 workers, configurable with `UV_THREADPOOL_SIZE`.

Q30. What happens when a Node.js application receives multiple requests?

Q31. What is the difference between CPU-bound and I/O-bound operations?

Node.js uses the **Event Loop** to handle multiple requests concurrently without creating a new JavaScript thread for every request. I/O operations are handled asynchronously, allowing Node.js to continue processing other requests.

Flow:

1. Requests arrive at the Node.js server.
2. Their JavaScript handlers execute on the **Call Stack**.
3. I/O operations are delegated to the OS/libuv.
4. While one request waits for I/O, Node.js can process other requests.
5. When I/O completes, its callback/Promise continuation is processed by the Event Loop.

Example:

```
app.get("/users", async (req, res) => {  
  const users = await getUsersFromDB();  
  res.json(users);  
});
```

While `getUsersFromDB( )` is waiting for the database:

Request A → Waiting for DB —————→ Response A
Request B → Processed
Request C → Processed
Request D → Processed

Key point:

Node.js achieves high concurrency by **not blocking the main JavaScript thread while waiting for I/O**.

Q32. Why are CPU-intensive operations problematic in a Node.js application?

CPU-intensive operations can **block Node.js's main JavaScript thread**, preventing the Event Loop from processing other requests and callbacks. This can cause **high latency and poor responsiveness**.

Why is it problematic?

1. JavaScript execution happens on the main thread.
2. CPU-heavy code keeps the **Call Stack busy**.
3. The Event Loop cannot process other callbacks.
4. Other requests have to wait.

Example:

```
app.get("/heavy", (req, res) => {  
  for (let i = 0; i < 1e10; i++) {  
    // CPU-intensive work  
  }  
}
```

```
  res.send("Done");  
});
```

While the loop runs:

CPU-heavy task



Main Thread blocked



Event Loop delayed



Other requests wait

How to handle it?

1. Use **Worker Threads** for CPU-heavy JavaScript.
2. Use **child processes / clustering** when appropriate.
3. Move heavy computation to a separate service.

Key point:

Node.js is excellent for **I/O-intensive workloads**, but CPU-heavy work can block the Event Loop unless it is offloaded.

Q33. What happens if you run a long-running synchronous operation in Node.js?

A long-running synchronous operation **blocks the main JavaScript thread and Event Loop** until it finishes. During that time, Node.js cannot process other callbacks or requests.

What happens?

1. The operation occupies the **Call Stack**.
2. Event Loop cannot process other tasks.
3. Incoming requests may have to wait.
4. Application latency increases and can appear **frozen/unresponsive**.

Example:

```
const fs = require("fs");  
  
const data = fs.readFileSync("large-file.txt");  
  
console.log("Done");
```

If reading the file takes a long time:

Synchronous operation

↓

Call Stack blocked

↓

Event Loop blocked

↓

Other requests wait

Better approach:

```
fs.readFile("large-file.txt", (err, data) => {  
  console.log("Done");  
});
```

Key point: Avoid long-running **synchronous operations** in request-handling code because they can block the Event Loop and affect **all clients**, not just the current request.

Q34. What does it mean when we say Node.js is event-driven?

Event-driven Node.js applications respond to **events** such as incoming HTTP requests, completed I/O operations, timers, or custom events. Handlers (callbacks/listeners) are executed when those events occur.

How it works:

1. An event occurs.
2. Node.js detects or receives the event.
3. The corresponding callback/listener is queued.
4. The Event Loop executes it when the Call Stack is available.

Example:

```
const EventEmitter = require("events");
```

```
const emitter = new EventEmitter();
```

```
emitter.on("login", () => {  
  console.log("User logged in");  
});
```

```
emitter.emit("login");
```

Flow:

Event occurs



Listener / Callback



Event Loop



Code executes

Key point: Node.js doesn't continuously wait for each operation to finish; it **reacts to events and executes the appropriate handlers**.

Q35. What is an event emitter in Node.js?

An **EventEmitter** is a Node.js mechanism that allows objects to **emit named events** and other parts of the application to **listen and respond** to those events.

It is provided by Node.js through the built-in **events** module.

Role of EventEmitter:

1. **emit()** → Triggers an event.
2. **on()** → Registers a listener for an event.
3. Enables **communication between different parts of an application**.

Example:

```
const { EventEmitter } = require("events");
```

```
const emitter = new EventEmitter();
```

```
emitter.on("login", (user) => {  
  console.log(`${user} logged in`);  
});
```

```
emitter.emit("login", "Alex");
```

Output:

Alex logged in

Key point: EventEmitter = emit events + listen for events, making it useful for event-driven communication within Node.js applications.

Q36. How does **EventEmitter** work?

EventEmitter maintains a list of **listeners (callbacks)** for named events. When an event is emitted using **emit()**, Node.js calls the registered listeners for that event.

How it works:

1. Create an **EventEmitter** instance.
2. Register listeners using **on()** or **once()**.
3. Trigger an event using **emit()**.
4. Registered listeners execute with the provided arguments.

Example:

```
const { EventEmitter } = require("events");
```

```
const emitter = new EventEmitter();
```

```
emitter.on("order", (id) => {  
  console.log(`Order received: ${id}`);  
});
```

```
emitter.emit("order", 101);
```

Flow:

```
on("order", callback)  
  ↓  
emit("order", 101)  
  ↓  
callback(101)  
  ↓  
"Order received: 101"
```

Useful methods:

Method	Purpose
on()	Add a listener
once()	Run listener only once
emit()	Trigger an event

<code>off() / removeListener()</code>	Remove a listener
---	-------------------

Key point:

`EventEmitter` follows a simple **publish/subscribe pattern**: one part emits an event, and registered listeners react to it.

Q37. What are events and listeners in Node.js?

An **event** is a named occurrence or action, while a **listener** is a callback function that waits for that event and executes when it occurs.

Role:

1. **Event** → Represents something that happened, e.g. `"login"`.
2. **Listener** → Handles the event.
3. `emit()` → Triggers the event.
4. `on()` → Registers a listener.

Example:

```
const { EventEmitter } = require("events");
```

```
const emitter = new EventEmitter();
```

```
emitter.on("login", (user) => {  
  console.log(`${user} logged in`);  
});
```

```
emitter.emit("login", "Alex");
```

Flow:

Event: "login"



Listener: callback



Execute handler

Key point:

Event = something that happens

Listener = function that responds to it.

Q38. How do you create and emit a custom event using `EventEmitter`?

You can create a custom event by creating an `EventEmitter` instance, registering a listener with `on()`, and triggering the event using `emit()`.

Steps:

1. Import `EventEmitter`.
2. Create an instance.
3. Register a listener using `on()`.
4. Emit the custom event using `emit()`.

Example:

```
const { EventEmitter } = require("events");

const emitter = new EventEmitter();

emitter.on("userRegistered", (name) => {
  console.log(`Welcome ${name}`);
});

emitter.emit("userRegistered", "Alex");
```

Output:

Welcome Alex

Key point:

`on()` **listens** for an event, while `emit()` **triggers** the event.

Q39. What is the difference between an event and a callback?

An **event** represents something that happens, while a **callback** is a function that is executed in response to an event or after an operation completes.

Feature	Event	Callback
Meaning	Represents an occurrence	Function executed later
Purpose	Signals that something happened	Handles the result/action
Relationship	Can trigger listeners	Can be the listener/handler
Example	"login" event	<pre>() => console.log("Logged in")</pre>

Example:

```
emitter.on("login", () => {
  console.log("User logged in"); // Callback
});
```

```
emitter.emit("login"); // Event
```

Here:

- "login" → **Event**
- () => { ... } → **Callback/Listener**
- emit("login") → **Triggers the event**

Key point:

Event = what happened

Callback = what to do when it happens

Q40. What is the purpose of the **process** object in Node.js?

The **process** object provides information and control over the **currently running Node.js process**. It is globally available, so you don't need to import it.

Role of **process**:

1. **Environment variables** → `process.env`
2. **Command-line arguments** → `process.argv`
3. **Process control** → `process.exit()`
4. **Process information** → `process.pid`, `process.version`
5. **Handle process events** → `process.on()`

Example:

```
console.log(process.pid);  
console.log(process.env.NODE_ENV);
```

```
process.on("exit", () => {  
  console.log("Process is exiting");  
});
```

Key point: The **process** object lets you **access information about and control the running Node.js process**.

Built-in APIs

Built-in APIs

Q41. What is `process.argv` used for?

`process.argv` is an array containing the **command-line arguments** passed when starting a Node.js application.

Role of `process.argv`:

1. Read arguments provided from the terminal.
2. Build simple CLI applications.
3. Pass configuration values to a Node.js script.

Example:

```
console.log(process.argv);
```

Run:

```
node app.js Alex 25
```

Output will contain:

```
[  
  ".../node",  
  ".../app.js",  
  "Alex",  
  "25"  
]
```

So:

```
const name = process.argv[2];  
console.log(`Hello ${name}`);
```

Output:

Hello Alex

Key point: `process.argv` allows Node.js programs to **receive input directly from the command line**.

Q42. What is `process.exit()` and when would you use it?

`process.exit()` immediately terminates the Node.js process with the specified exit code. `0` generally indicates success, while a non-zero code indicates an error or abnormal termination.

When to use it:

1. Exit after a **fatal/ unrecoverable error**.
2. Stop a CLI program when required input is missing.
3. Return an appropriate exit status to the operating system.

Example:

```
if (!process.env.DATABASE_URL) {  
  console.error("DATABASE_URL is missing");  
  process.exit(1);  
}
```

Here, `1` indicates that the process exited because of an error.

Key point: `process.exit()` should be used carefully because it **terminates immediately**, potentially interrupting pending asynchronous operations. Prefer allowing Node.js to exit naturally when possible.

Q43. What is `process.cwd()`?

`process.cwd()` returns the **current working directory** from which the Node.js process was started. It is not necessarily the directory where the JavaScript file is located.

Role of `process.cwd()`:

1. Get the application's current working directory.
2. Build paths relative to where the process was started.
3. Useful for CLI tools and file operations.

Example:

```
console.log(process.cwd());
```

If you run:

```
cd /projects/my-app  
node src/server.js
```

Output:

/projects/my-app

Key point:

`process.cwd()` → where the Node.js process was started
`__dirname` → where the current file is located (CommonJS).

Q44. What is the difference between `__dirname` and `__filename`?

Both are **CommonJS module variables** in Node.js. `__dirname` gives the directory of the current file, while `__filename` gives the **absolute path of the current file**.

Feature	<code>__dirname</code>	<code>__filename</code>
Returns	Directory path	File path
Includes filename	No	Yes
Example	<code>/app/src</code>	<code>/app/src/server.js</code>
Common use	Build paths to files	Identify the current file

Example:

```
console.log(__dirname);  
console.log(__filename);
```

Output:

```
/app/src  
/app/src/server.js
```

Example usage:

```
const path = require("path");  
  
const file = path.join(__dirname, "data.json");
```

Key point: `__dirname` → directory containing the current file
`__filename` → absolute path of the current file.

Q45. What is the **fs** module used for?

The **fs** (**File System**) module is a built-in Node.js module used to **read, write, modify, delete, and manage files and directories**.

Role of **fs**:

1. Read and write files.
2. Create and delete files/directories.
3. Rename and move files.
4. Check file information and permissions.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

Key point:

fs allows Node.js applications to **interact with the file system**. It provides both synchronous and asynchronous APIs.

Q46. What is the difference between synchronous and asynchronous file-system methods?

Synchronous (Sync) methods block the Node.js main thread until the file operation finishes, while **asynchronous methods** allow Node.js to continue executing other work.

Feature	Synchronous	Asynchronous
Blocks Event Loop	Yes	No
Execution	Waits for completion	Continues immediately
Performance	Can reduce concurrency	Better for servers
Example	<code>fs.readFileSync()</code>	<code>fs.readFile()</code>

Best use	Simple scripts/CLI	Web servers & APIs
----------	--------------------	--------------------

Synchronous:

```
const data = fs.readFileSync("data.txt", "utf8");
console.log(data);
console.log("Done");
```

Asynchronous:

```
fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});

console.log("Done");
```

Async output:

```
Done
[file content]
```

Key point:

For backend/server code, prefer **asynchronous file-system methods** because synchronous operations can block the Event Loop.

Q47. How do you read a file using Node.js?

You can read a file using the built-in **fs module**. The preferred approach for server applications is the **asynchronous fs.readFile()** method.

Steps:

1. Import the **fs** module.
2. Call **fs.readFile()**.
3. Handle the result in the callback.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

```
}  
  
console.log(data);  
});
```

Using **async/await**:

```
const fs = require("fs/promises");  
  
async function readFile() {  
  const data = await fs.readFile("data.txt", "utf8");  
  console.log(data);  
}
```

Key point:

Use **fs.readFile()** or **fs/promises.readFile()** for **non-blocking file reading** in Node.js applications.

Q48. How do you write data to a file using Node.js?

You can write data to a file using the built-in **fs module**. The asynchronous **fs.writeFile()** method is commonly used because it doesn't block the Event Loop.

Steps:

1. Import the **fs** module.
2. Call **fs.writeFile()**.
3. Handle success or errors in the callback.

Example:

```
const fs = require("fs");  
  
fs.writeFile("data.txt", "Hello Node.js", (err) => {  
  if (err) {  
    console.error(err);  
    return;  
  }  
  
  console.log("File written successfully");  
});
```

Using **async/await**:

```
const fs = require("fs/promises");  
  
await fs.writeFile("data.txt", "Hello Node.js");
```

Key point:

writeFile() creates the file if it doesn't exist and overwrites it if it does. Use **appendFile()** when you want to add data without overwriting existing content.

Q49. What is the **path** module, and why is it useful?

The **path** module is a built-in Node.js module used to **work with file and directory paths**. It provides platform-independent methods for creating, resolving, and analyzing paths.

Why is it useful?

1. Build paths safely across Windows, Linux, and macOS.
2. Join multiple path segments.
3. Extract file names, extensions, and directories.
4. Resolve relative paths into absolute paths.

Example:

```
const path = require("path");  
  
const filePath = path.join(__dirname, "data", "users.json");  
  
console.log(filePath);
```

Common methods:

Method	Purpose
path.join()	Joins path segments
path.resolve()	Creates an absolute path
path.basename()	Gets the file name
path.dirname()	Gets the directory

<code>path.extname()</code>	Gets the file extension
-----------------------------	-------------------------

Key point:

`path` helps you handle file paths **safely and consistently across different operating systems**.

Q50. What is the difference between a relative path and absolute path?

An **absolute path** specifies the complete location of a file from the root of the file system, while a **relative path** specifies a location relative to the current working directory or another reference location.

Feature	Absolute Path	Relative Path
Starts from	File-system root	Current/reference directory
Complete location	Yes	No
Depends on current directory	No	Yes
Example	<code>/home/app/data.txt</code>	<code>./data.txt</code>
Portability	Less portable	Usually more portable

Example:

```
const path = require("path");

console.log(path.resolve("data.txt"));
// /home/app/data.txt

console.log(path.join(".", "data.txt"));
// data.txt
```

Key point:

Absolute path = complete location

Relative path = location relative to a reference directory.

Q51. What is the **http** module in Node.js?

The **http** module is a built-in Node.js module used to **create HTTP servers and handle HTTP requests and responses** without requiring an external framework.

Role of **http**:

1. Create HTTP servers.
2. Handle incoming requests.
3. Send HTTP responses.
4. Access request details such as URL and HTTP method.

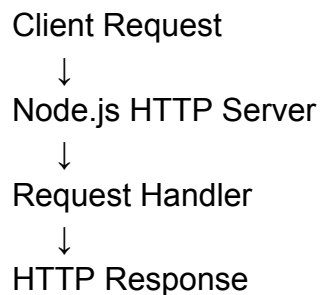
Example:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello Node.js");
});

server.listen(3000);
```

Flow:



Key point:

The **http** module provides the **low-level HTTP functionality** that frameworks like Express build upon.

Q52. How can you create an HTTP server using Node.js without Express?

You can create an HTTP server using Node.js's built-in **http module**. The `http.createServer()` method receives a callback that handles incoming requests and sends responses.

Steps:

1. Import the `http` module.
2. Create a server using `http.createServer()`.
3. Handle the request and response.
4. Start the server using `listen()`.

Example:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello World!");
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Flow:

```
Client
↓
HTTP Server
↓
Request Handler
↓
Response
```

Key point:

Node.js can create HTTP servers **without Express** using its built-in `http` module; Express mainly provides a higher-level, more convenient API for routing and middleware.

Q53. What are streams in Node.js?

Streams are objects that allow Node.js to **read or write data gradually in chunks**, instead of loading the entire data into memory at once. They are useful for large files, network requests, and real-time data.

Types of Streams:

1. **Readable** → Read data, e.g. `fs.createReadStream()`.
2. **Writable** → Write data, e.g. `fs.createWriteStream()`.
3. **Duplex** → Both readable and writable.
4. **Transform** → Modify data while reading/writing.

Example:

```
const fs = require("fs");
```

```
const stream = fs.createReadStream("large-file.txt", "utf8");
```

```
stream.on("data", (chunk) => {  
  console.log(chunk);  
});
```

Instead of loading the entire file:

Large File

↓

[Chunk 1] → Process

[Chunk 2] → Process

[Chunk 3] → Process

Key point:

Streams improve **memory efficiency and performance** when handling large amounts of data.

Q54. What is the difference between a stream and reading an entire file at once?

A **stream** processes data in **small chunks**, while methods like `fs.readFile()` load the **entire file into memory** before processing it.

Feature	Stream	Entire File
Data handling	Chunk by chunk	Entire file
Memory usage	Low	Higher
Large files	Better	Can be problematic
Processing	Can start immediately	Starts after entire file is loaded
Example	<code>createReadStream()</code>	<code>readFile()</code>

Stream:

```
const stream = fs.createReadStream("large.txt");
```

```
stream.on("data", chunk => {  
  console.log(chunk);  
});
```

Entire file:

```
fs.readFile("large.txt", (err, data) => {  
  console.log(data);  
});
```

Example:

For a **1 GB file**, `readFile()` attempts to load the whole file into memory, while a stream processes it in smaller chunks.

Key point:

Streams → better for large/continuous data
`readFile()` → simpler for small files.

Q55. What are the different types of streams in Node.js?

Node.js provides **four main types of streams**, each designed for a different way of handling data.

Types of Streams:

1. **Readable** → Used to read data.
 - Example: `fs.createReadStream()`
2. **Writable** → Used to write data.
 - Example: `fs.createWriteStream()`
3. **Duplex** → Can both read and write data.
 - Example: TCP socket
4. **Transform** → A type of Duplex stream that can modify data while it passes through.
 - Example: `zlib.createGzip()`

Stream	Purpose	Example
Readable	Read data	<code>createReadStream()</code>
Writable	Write data	<code>createWriteStream()</code>
Duplex	Read + Write	TCP socket
Transform	Read + Write + Transform	<code>createGzip()</code>

Example:

```
const fs = require("fs");

const read = fs.createReadStream("input.txt");
const write = fs.createWriteStream("output.txt");

read.pipe(write);
```

Here, data is **read in chunks and written to another file** without loading the entire file into memory.

Key point:

Readable → Read | **Writable** → Write | **Duplex** → Read + Write |
Transform → Read + Write + Modify

Q56. What is a Buffer in Node.js?

A **Buffer** is a Node.js object used to work with **raw binary data** directly in memory. It is commonly used when handling files, network data, streams, and other binary data.

Role of Buffer:

1. Store binary data as bytes.
2. Handle data received from streams and network connections.
3. Convert between strings and binary data.

Example:

```
const buffer = Buffer.from("Hello");
```

```
console.log(buffer);  
// <Buffer 48 65 6c 6c 6f>
```

```
console.log(buffer.toString());  
// Hello
```

Common methods:

Method	Purpose
<code>Buffer.from()</code>	Create buffer from data
<code>Buffer.alloc()</code>	Create an initialized buffer
<code>.toString()</code>	Convert buffer to string

Key point:

Buffer = temporary memory for handling raw binary data, especially when working with streams and I/O.

Q57. Why does Node.js need Buffers?

Node.js needs **Buffers** because many I/O operations work with **raw binary data** rather than JavaScript strings. Buffers provide a way to efficiently store and process these bytes in memory.

Why Buffers are needed:

1. Handle **binary data** from files and network connections.
2. Process data received through **streams** in chunks.
3. Store data efficiently as **raw bytes**.
4. Convert between binary data and strings.

Example:

```
const fs = require("fs");

fs.readFile("image.png", (err, data) => {
  console.log(data); // Buffer containing binary data
});
```

For an image, Node.js cannot treat the content as normal text. The **Buffer** stores the **raw bytes** of the image.

Key point:

Buffers allow Node.js to efficiently handle raw binary data during I/O operations.

Q58. What is the difference between a Buffer and a string?

A **String** represents text using characters, while a **Buffer** represents raw binary data as bytes. Buffers are commonly used for files, streams, and network data.

Feature	Buffer	String
Represents	Binary data / bytes	Text / characters
Data type	Raw bytes	Unicode text
Common use	Files, streams, network data	Text processing
Encoding	Can be encoded/decoded	Uses character encoding internally

Example	<code>Buffer.from("Hello")</code>	<code>"Hello"</code>
---------	-----------------------------------	----------------------

Example:

```
const buffer = Buffer.from("Hello");
const string = "Hello";
```

```
console.log(buffer);
// <Buffer 48 65 6c 6c 6f>
```

```
console.log(string);
// Hello
```

Conversion:

```
const buffer = Buffer.from("Hello");
const text = buffer.toString("utf8");
```

Key point:

String → **text/characters**

Buffer → **raw bytes/binary data.**

Q59. What is the `url` module used for?

The built-in `url` module is used to **parse, construct, and manipulate URLs** in Node.js. It is especially useful for extracting parts like the hostname, pathname, and query parameters.

Role of `url`:

1. Parse URLs into individual components.
2. Extract query parameters.
3. Construct and modify URLs.
4. Convert URL objects to strings.

Example:

```
const { URL } = require("url");
```

```
const url = new URL(
  "https://example.com/users?id=10"
);
```

```
console.log(url.pathname); // /users
console.log(url.searchParams.get("id")); // 10
```

Common properties:

Property	Purpose
hostname	Domain name
pathname	Path of the URL
searchParams	Query parameters
protocol	http: / https:

Key point:

The `url` module helps Node.js applications **parse and work with URLs and query parameters**.

Q60. What is the `crypto` module used for?

The built-in **`crypto` module** provides cryptographic functionality in Node.js. It is used for **hashing, encryption/decryption, generating secure random values, and digital signatures**.

Role of `crypto`:

1. **Hashing** → Create hashes using algorithms like SHA-256.
2. **Password hashing** → `scrypt()`, `pbkdf2()`.
3. **Encryption/Decryption** → Protect sensitive data.
4. **Random values** → Generate secure tokens/keys.

Example:

```
const crypto = require("crypto");
```

```
const hash = crypto
  .createHash("sha256")
  .update("Hello")
  .digest("hex");
```

```
console.log(hash);
```

Key point:

`crypto` provides Node.js with **secure cryptographic operations** without requiring an external package.

Error Handling

Error Handling

Q61. How does error handling work in Node.js?

Node.js handles errors differently depending on whether the code is **synchronous** or **asynchronous**. Common approaches include `try/catch`, callback error arguments, Promise `.catch()`, and `async/await` with `try/catch`.

Common approaches:

1. **Synchronous** → Use `try/catch`.
2. **Callbacks** → Use the error-first callback pattern `(err, data)`.
3. **Promises** → Use `.catch()`.
4. **Async/Await** → Use `try/catch`.
5. **EventEmitter** → Handle the special `'error'` event.

Example with `async/await`:

```
async function getData() {
  try {
    const data = await fetchData();
    console.log(data);
  } catch (error) {
    console.error("Error:", error.message);
  }
}
```

Callback example:

```
fs.readFile("data.txt", (err, data) => {
  if (err) {
    console.error(err);
    return;
  }

  console.log(data);
});
```

Key point:

Always handle errors appropriately, especially in asynchronous code, to prevent unexpected application crashes.

Q62. What is the difference between operational errors and programming errors?

Operational errors are expected runtime problems that can often be handled gracefully, while **programming errors** are bugs in the code that usually require fixing the application.

Feature	Operational Error	Programming Error
Meaning	Runtime/environment problem	Bug in application code
Usually expected?	Yes	No
Can often recover?	Yes	Usually requires a fix
Examples	DB unavailable, timeout, invalid user input	<code>TypeError</code> , undefined variable, incorrect logic
Handling	Retry, fallback, proper response	Fix the underlying code

Operational error:

```
try {  
  await connectToDatabase();  
} catch (err) {  
  console.log("Database unavailable");  
}
```

The database being temporarily unavailable may be an expected runtime condition.

Programming error:

```
const user = null;  
  
console.log(user.name); // TypeError
```

This indicates a bug that should be fixed.

Key point:

Operational error → handle/recover when possible
Programming error → identify and fix the code.

Q63. What is the purpose of `try...catch` in Node.js?

`try...catch` is used to **catch and handle errors** that occur during synchronous execution and errors from awaited Promises inside an `async` function.

Role of `try...catch`:

1. Prevent unhandled exceptions from propagating.
2. Handle errors gracefully.
3. Provide fallback logic or meaningful error messages.

Example:

```
try {
  const data = JSON.parse("invalid-json");
  console.log(data);
} catch (error) {
  console.log("Invalid JSON");
}
```

With `async/await`:

```
async function getUser() {
  try {
    const user = await fetchUser();
    return user;
  } catch (error) {
    console.error("Failed:", error.message);
  }
}
```

Key point:

`try...catch` catches **synchronous exceptions** and rejected Promises when they are **awaited inside the `try` block**.

Q64. How do you handle errors in asynchronous Node.js code?

The approach depends on the asynchronous pattern being used. With modern Node.js, **Promises** and **async/await** with **try...catch** are commonly preferred.

Common approaches:

1. **Callbacks** → Check the **err** parameter.
2. **Promises** → Use **.catch()**.
3. **async/await** → Use **try...catch**.
4. **EventEmitter** → Handle the **'error'** event.

Callback:

```
fs.readFile("data.txt", (err, data) => {  
  if (err) return console.error(err);  
  console.log(data);  
});
```

Promise:

```
getData()  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Async/Await:

```
try {  
  const data = await getData();  
  console.log(data);  
} catch (err) {  
  console.error(err);  
}
```

Key point:

Always handle asynchronous failures explicitly to avoid **unhandled errors or rejected Promises**.

Q65. What happens when an uncaught exception occurs?

An **uncaught exception** is an error that is thrown but not handled by any `try...catch` or other appropriate error-handling mechanism. In Node.js, this can cause the **process to terminate**.

What happens?

1. Node.js detects the uncaught exception.
2. The error is reported, usually with a stack trace.
3. If not handled, the Node.js process **exits**.
4. Pending work may be interrupted.

Example:

```
throw new Error("Something went wrong");
```

```
console.log("This won't execute");
```

Output:

```
Error: Something went wrong  
    at ...
```

Handling it globally:

```
process.on("uncaughtException", (err) => {  
  console.error("Uncaught:", err);  
});
```

However, this should **not be treated as a normal recovery mechanism**. After an uncaught exception, the application may be in an inconsistent state, so the safer approach is usually to **log the error, shut down gracefully, and restart the process**.

Key point:

Uncaught exception → process can terminate → application should be restarted safely.

Q66. What happens when an unhandled Promise rejection occurs?

An **unhandled Promise rejection** occurs when a Promise is rejected and no handler such as `.catch()` or `try...catch` with `await` handles the rejection.

In modern Node.js, an unhandled rejection is generally treated as an **uncaught exception and can terminate the process**.

Example:

```
Promise.reject(new Error("Database failed"));
```

If there is no rejection handler, Node.js reports an **unhandled rejection** and may terminate the process.

Handle it with `.catch()`:

```
getData()  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Or with `async/await`:

```
try {  
  const data = await getData();  
} catch (err) {  
  console.error(err);  
}
```

Key point:

Always handle Promise rejections to avoid **unexpected process termination and unreliable application behavior**.

Q67. What is the difference between `throw new Error()` and returning an error?

`throw new Error()` immediately interrupts the current execution flow and propagates an exception to the nearest error handler. Returning an error simply passes a value back to the caller, which must explicitly check it.

Feature	<code>throw new Error()</code>	Return an error
Flow	Immediately interrupts execution	Continues normally
Handling	<code>try...catch</code> / error handler	Caller checks the returned value
Typical use	Unexpected/failure conditions	Expected result/status
Example	<code>throw new Error("Failed")</code>	<code>return { error: "Failed" }</code>

Using `throw`:

```
function getUser() {  
  throw new Error("User not found");  
}
```

```
try {  
  getUser();  
} catch (err) {  
  console.log(err.message);  
}
```

Returning an error:

```
function getUser() {  
  return { error: "User not found" };  
}
```

```
const result = getUser();
```

```
if (result.error) {  
  console.log(result.error);  
}
```

Key point:

`throw` → exceptional failure that should be handled

`return` → normal value that the caller explicitly handles.

Q68. How do you debug a Node.js application?

Node.js provides several tools for debugging, from simple console logging to the built-in **Node.js Inspector** and Chrome DevTools/VS Code debugger.

Common debugging methods:

1. **console.log()** → Quickly inspect values and program flow.
2. **Node Inspector** → Debug with breakpoints and step-through execution.
3. **VS Code Debugger** → Set breakpoints, inspect variables, and view call stacks.
4. **Error stack traces** → Identify where an error occurred.
5. **--inspect** → Enable the Node.js debugging protocol.

Example:

```
function add(a, b) {  
  debugger;  
  return a + b;  
}
```

```
console.log(add(10, 20));
```

Run with:

```
node --inspect app.js
```

You can then connect a debugger and inspect execution.

Key point:

For quick debugging use **console.log()**, while for complex issues use **breakpoints, stack traces, and the Node.js Inspector**.

Q69. What is nodemon, and why is it useful during development?

Nodemon is a development tool that automatically **restarts a Node.js application whenever it detects changes in the source files**. This eliminates the need to manually stop and restart the server.

Why is it useful?

1. **Automatic restart** → Restarts the server after file changes.
2. **Faster development** → No manual server restarts.
3. **Improves workflow** → Lets you immediately test code changes.

Example:

Install:

```
npm install --save-dev nodemon
```

Add to `package.json`:

```
{  
  "scripts": {  
    "dev": "nodemon server.js"  
  }  
}
```

Run:

```
npm run dev
```

Now, whenever `server.js` changes, Nodemon automatically restarts the application.

Key point:

Nodemon is a development-only tool that automatically restarts your Node.js server when files change.

Q70. How would you troubleshoot a Node.js application that is crashing or becoming unresponsive?

I would first identify **whether the issue is a crash, Event Loop blockage, memory problem, or external dependency failure**, then use logs and debugging tools to find the root cause.

Troubleshooting steps:

1. **Check logs & stack traces** → Identify uncaught exceptions and errors.
2. **Check Event Loop blocking** → Look for CPU-intensive or synchronous operations.
3. **Check memory usage** → Look for memory leaks or excessive heap usage.
4. **Check external services** → Database, APIs, network connections, etc.

5. **Use profiling/debugging tools** → Node Inspector, CPU profiles, heap snapshots.
6. **Monitor production metrics** → CPU, memory, latency, error rate, and request throughput.

Example:

```
console.time("operation");
```

```
// Suspected slow operation  
heavyOperation();
```

```
console.timeEnd("operation");
```

For a memory issue, you can inspect the process:

```
node --inspect app.js
```

Quick diagnosis:

Symptom	Possible Cause
Process crashes	Uncaught exception / fatal error
Requests hang	Event Loop blocked
Memory keeps increasing	Memory leak
CPU near 100%	CPU-intensive operation
Slow API responses	DB/network/CPU bottleneck

Key point:

Start with **logs and metrics**, identify the bottleneck, reproduce it, then use profiling/debugging tools to confirm the root cause.

Express.js

Express.js Fundamentals

Check sub-tabs for complete QnA

Basics & Setup

Basics & Setup

Q1. What is Express.js, and why do we use it with Node.js?

Express.js is a lightweight and flexible **web framework for Node.js** used to build web servers, REST APIs, and backend applications. It simplifies Node.js's built-in HTTP functionality.

Why use Express.js?

1. **Routing** → Easily define endpoints like `GET /users`.
2. **Middleware** → Handle authentication, logging, validation, etc.
3. **Request/Response handling** → Simplifies working with HTTP requests and responses.
4. **Error handling** → Provides structured error-handling mechanisms.
5. **Faster development** → Less boilerplate than using Node's `http` module directly.

Example:

```
const express = require("express");

const app = express();

app.get("/users", (req, res) => {
  res.json({ message: "Users fetched" });
});

app.listen(3000);
```

Key point: Node.js provides the runtime and HTTP capabilities; Express.js provides a simpler framework for building web servers and APIs.

Q2. How do you create a basic Express server?

A basic Express server is created by installing Express, creating an Express application, defining routes, and starting the server with `listen()`.

Steps:

1. Install Express.
2. Import `express`.
3. Create an Express app.
4. Define a route.
5. Start the server.

Example:

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.send("Hello Express!");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Run:

```
node server.js
```

Then visit:

```
http://localhost:3000
```

Key point:

`express()` creates the application, routes handle requests, and `listen()` starts the server.

Q3. What is a route in Express?

A **route** defines how an Express application responds to a specific **HTTP method** and **URL path**. It connects an incoming request to a handler function.

Role of a Route:

1. Defines the **HTTP method** → **GET**, **POST**, **PUT**, **DELETE**, etc.
2. Defines the **URL path** → **/users**, **/products/:id**, etc.
3. Executes a **handler** when the request matches.

Example:

```
app.get("/users", (req, res) => {  
  res.json({ message: "Users fetched" });  
});
```

Here:

- **GET** → HTTP method
- **/users** → Route path
- **(req, res) => { ... }** → Route handler

Key point:

Route = HTTP method + URL path + handler that determines how Express responds to a request.

Q4. What is the difference between GET, POST, PUT, PATCH, and DELETE?

These are HTTP methods used to perform different operations on resources in API.

Method	Purpose	Example	Typical Operation
GET	Retrieve data	GET /users	Read
POST	Create a new resource	POST /users	Create
PUT	Replace/update a resource completely	PUT /users/1	Full Update
PATCH	Partially update a resource	PATCH /users/1	Partial Update

DELETE	Remove a resource	DELETE <code>/users/1</code>	Delete
---------------	-------------------	--	--------

Example:

```
app.get("/users", getUsers);
app.post("/users", createUser);
app.put("/users/:id", replaceUser);
app.patch("/users/:id", updateUser);
app.delete("/users/:id", deleteUser);
```

Simple analogy:

```
GET    → Give me the user
POST   → Create a new user
PUT    → Replace the user
PATCH → Change part of the user
DELETE → Remove the user
```

Key point:

PUT generally replaces the **entire resource**

PATCH modifies **only specific fields**.

Q5. What is middleware in Express?

Middleware is a function that runs **between receiving an HTTP request and sending the response**. It can inspect/modify the request and response, perform actions, or pass control to the next middleware using `next()`.

Role of Middleware:

1. **Authentication/Authorization** → Verify users.
2. **Logging** → Track incoming requests.
3. **Validation** → Validate request data.
4. **Parsing** → Parse JSON/body data.
5. **Error handling** → Handle application errors.

Example:

```
const logger = (req, res, next) => {
  console.log(req.method, req.url);
  next();
};

app.use(logger);

app.get("/users", (req, res) => {
  res.json({ message: "Users" });
});
```

Flow:

```
Request
  ↓
Middleware
  ↓ next()
Route Handler
  ↓
Response
```

Key point:

Middleware can execute code, modify **req/res**, end the request, or call **next()** to continue the request pipeline.

Q6. What is the difference between application-level and route-level middleware?

Application-level middleware is attached to the Express app using **app.use()** or **app.METHOD()** and can apply to **many or all routes**. **Route-level middleware** is attached to a specific route or router and runs only for those routes.

Feature	Application-Level	Route-Level
Scope	Entire application / route group	Specific route(s)
Attached using	app.use()	Route/Router

Use case	Logging, CORS, body parsing	Authentication, validation
Example	<code>app.use(logger)</code>	<code>app.get("/admin", auth, handler)</code>

Application-level:

```
app.use((req, res, next) => {
  console.log(req.method);
  next();
});
```

Runs for requests that match its mounted path.

Route-level:

```
const auth = (req, res, next) => {
  next();
};

app.get("/admin", auth, (req, res) => {
  res.send("Admin");
});
```

Here, `auth` runs **only for `/admin`**.

Key point:

Application-level → broader scope

Route-level → specific route or router.

Q7. What does `next()` do in Express middleware?

`next()` passes control from the **current middleware to the next matching middleware or route handler** in the Express request-response cycle.

Role of `next()`:

1. Continues the middleware chain.
2. Allows the next handler to execute.
3. `next(error)` passes an error to Express's error-handling middleware.

Example:

```
app.use((req, res, next) => {
  console.log("Middleware 1");
  next();
});

app.get("/", (req, res) => {
  res.send("Hello");
});
```

Flow:

```
Request
  ↓
Middleware 1
  ↓ next()
Route Handler
  ↓
Response
```

Important: If middleware neither **sends a response** nor calls `next()`, the request can remain hanging.

Key point:

`next()` means "I'm done; continue to the next matching handler."

Q8. In what order does Express execute middleware and routes?

Express executes middleware and routes **in the order they are registered in the application**. When a request arrives, Express moves through the matching middleware/route stack sequentially.

Execution Order:

1. Global/application middleware.
2. Router-level middleware.
3. Route-specific middleware.
4. Route handler.
5. Error-handling middleware, if an error is passed using `next(error)`.

Example:

```
app.use((req, res, next) => {
  console.log("1");
  next();
});

app.use((req, res, next) => {
  console.log("2");
  next();
});

app.get("/", (req, res) => {
  console.log("3");
  res.send("Done");
});
```

Output:

```
1
2
3
```

Important: If middleware sends a response or doesn't call `next()`, the subsequent handlers **won't execute**.

Key point:

Express follows a **top-to-bottom, registration-order pipeline**, executing each matching middleware/route until the response is completed.

Q9. How do you handle JSON data sent in an Express request?

Use Express's built-in `express.json()` middleware to parse incoming JSON request bodies. The parsed data becomes available through `req.body`.

Steps:

1. Add `express.json()` middleware.
2. Send JSON in the request body.
3. Access it using `req.body`.

Example:

```
const express = require("express");
const app = express();

app.use(express.json());

app.post("/users", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
```

Request:

```
{
  "name": "Alex",
  "age": 22
}
```

`req.body`:

```
{
  name: "Alex",
  age: 22
}
```

Key point:

`express.json()` parses JSON request bodies and makes the resulting object available through `req.body`.

Q10. What is the difference between `req.params`, `req.query`, and `req.body`?

These are three ways Express provides **data sent by the client**. The main difference is **where the data comes from in the HTTP request**.

Feature	<code>req.params</code>	<code>req.query</code>	<code>req.body</code>
Source	URL path	URL query string	Request body

Example URL	<code>/users/123</code>	<code>/users?page=2</code>	<code>POST /users</code>
Access	<code>req.params.id</code>	<code>req.query.page</code>	<code>req.body.name</code>
Common use	Resource identification	Filtering, sorting, pagination	Create/update data

Example:

```
app.post("/users/:id", (req, res) => {
  console.log(req.params.id); // 123
  console.log(req.query.role); // admin
  console.log(req.body.name); // Alex
});
```

Reque

```
st:
POST /users/123?role=admin
Body: { "name": "Alex" }
```

Simple rule:

```
req.params → "Which resource?"
req.query → "How should I fetch it?"
req.body → "What data am I sending?"
```

Key point:

```
params → path data
query → URL query data
body → request payload
```

Requests & Responses

Requests & Responses

Q11. How do you send a response from an Express route?

Express provides methods on the **res (response) object** to send data back to the client. Common methods include `res.send()`, `res.json()`, `res.status()`, and `res.end()`.

Common methods:

1. `res.send()` → Send text, HTML, or other data.
2. `res.json()` → Send a JSON response.
3. `res.status()` → Set the HTTP status code.
4. `res.end()` → End the response without sending a body.

Example:

```
app.get("/users", (req, res) => {  
  res.status(200).json({  
    name: "Alex",  
    role: "Developer"  
  });  
});
```

Other examples:

```
res.send("Hello");  
res.json({ message: "Success" });  
res.status(404).json({ error: "Not found" });
```

Key point:

Use `res.json()` for APIs, `res.send()` for general responses, and `res.status()` to set the appropriate HTTP status code.

Q12. What is an HTTP status code? Explain common codes such as 200, 201, 400, 401, 403, 404, and 500.

An **HTTP status code** is a 3-digit code sent by the server to indicate the **result of a client's request**. It tells the client whether the request succeeded, failed, or requires further action.

Code	Meaning	Common Use
200	OK	Request successful
201	Created	Resource successfully created
400	Bad Request	Invalid/malformed client input
401	Unauthorized	Authentication is missing/invalid
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource/route doesn't exist
500	Internal Server Error	Unexpected server-side error

Example:

```
app.post("/users", (req, res) => {  
  const user = createUser(req.body);  
  
  res.status(201).json(user);  
});
```

Here, **201 Created** tells the client that a new user was successfully created.

Key distinction:

```
401 → "Who are you?"  
403 → "I know who you are, but you're not allowed."
```

Key point:

Use status codes correctly so clients can **understand the outcome of an API request** without relying only on the response body.

Q13. What is error-handling middleware in Express?

Error-handling middleware is special Express middleware used to **catch and handle errors** that occur during request processing. It is identified by having **four parameters**: `(err, req, res, next)`.

Role of Error-Handling Middleware:

1. Catch errors passed using `next(error)`.
2. Send a consistent error response to the client.
3. Log errors for debugging/monitoring.
4. Prevent exposing sensitive internal details.

Example:

```
app.get("/users", (req, res, next) => {
  try {
    throw new Error("Database failed");
  } catch (err) {
    next(err);
  }
});

app.use((err, req, res, next) => {
  console.error(err);

  res.status(500).json({
    error: "Internal Server Error"
  });
});
```

Flow:

```
Route
↓
next(error)
↓
Error Middleware
↓
HTTP Response
```

Key point: Express error-handling middleware has the signature `(err, req, res, next)` and should generally be registered **after your routes and other middleware**.

Q14. How do you create a global error handler in Express?

A **global error handler** is an error-handling middleware registered at the application level. It catches errors passed through `next(error)` and sends a consistent response.

Steps:

1. Define your routes/middleware.
2. Pass errors using `next(error)`.
3. Add one error-handling middleware **after all routes**.
4. Send an appropriate status and error response.

Example:

```
const express = require("express");
const app = express();

app.get("/users", (req, res, next) => {
  next(new Error("Database failed"));
});

// Global error handler
app.use((err, req, res, next) => {
  console.error(err);

  res.status(500).json({
    message: "Internal Server Error"
  });
});

app.listen(3000);
```

Flow:

```
Request
  ↓
Route
```

```
↓
next(error)
↓
Global Error Handler
↓
500 Response
```

Key point:

A global error handler uses `(err, req, res, next)` and should be registered **after all routes and other middleware**.

Q15. How do you organize routes and controllers in an Express application?

A common approach is to separate **routing logic** from **business logic**. Routes define the endpoints, while controllers contain the logic for handling those requests.

Typical structure:

```
src/
├── routes/
│   └── user.routes.js
├── controllers/
│   └── user.controller.js
├── models/
│   └── user.model.js
├── middleware/
│   └── auth.js
└── app.js
```

Route:

```
// user.routes.js
const router = require("express").Router();
const { getUsers } = require("../controllers/user.controller");

router.get("/users", getUsers);

module.exports = router;
```

Controller:

```
// user.controller.js
exports.getUsers = (req, res) => {
  res.json({ users: [] });
};
```

Register route:

```
// app.js
app.use("/api", userRoutes);
```

Benefits:

1. **Separation of concerns** → Routes and business logic stay separate.
2. **Maintainability** → Easier to modify and debug.
3. **Reusability** → Controllers can be reused by multiple routes.
4. **Scalability** → Easier to organize large applications.

Key point: Routes decide "which endpoint?" → Controllers decide "what should happen?"

Q16. What is CORS, and why does a MERN application need it?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that allows a server to specify which **different origins** are permitted to access its resources. Browsers normally restrict cross-origin requests because of the **same-origin policy**.

In a MERN application, the **React frontend and Express/Node.js backend often run on different origins** during development, so CORS allows the frontend to communicate with the backend API.

Example:

```
React:    http://localhost:5173
          ↓
        CORS
          ↓
Express:  http://localhost:5000
```

Enable CORS in Express:

```
const cors = require("cors");

app.use(cors({
  origin: "http://localhost:5173"
}));
```

The server then sends headers such as `Access-Control-Allow-Origin` to tell the browser which origin is allowed.

Key point: CORS allows controlled cross-origin communication between the React frontend and Express backend.

Q17. What is the difference between authentication and authorization?

Authentication verifies **who the user is**, while **authorization** determines **what that authenticated user is allowed to do**.

Feature	Authentication	Authorization
Question	"Who are you?"	"What can you access?"
Purpose	Verify identity	Check permissions
Happens	Usually first	After authentication
Example	Login with email/password	Admin can delete users
Common mechanisms	Passwords, JWT, OAuth	Roles, permissions, policies

Example:

```
// Authentication
if (!user) return res.status(401).json({ error: "Not authenticated" });

// Authorization
if (user.role !== "admin") {
  return res.status(403).json({ error: "Forbidden" });
}
```

```
}
```

Simple analogy:

Authentication → Prove your identity → "You are Alex"

Authorization → Check permissions → "You can access /admin"

Key point:

Authentication = Who are you?

Authorization = What are you allowed to do?

Q18. What is the difference between `res.send()`, `res.json()`, and `res.status()`?

These are Express response methods used to **send data and control the HTTP response**.

Method	Purpose	Example
<code>res.send()</code>	Sends a response body (text, HTML, object, etc.)	<code>res.send("Hello")</code>
<code>res.json()</code>	Sends a JSON response	<code>res.json({ user: "Alex" })</code>
<code>res.status()</code>	Sets the HTTP status code	<code>res.status(404)</code>

Examples:

```
res.send("Hello World");

res.json({ message: "Success" });

res.status(404).json({
  error: "User not found"
});
```

You can **chain** `status()` with `json()` or `send()`:

```
res.status(201).json({
  message: "User created"
});
```

Key point:

`res.send()` → **send response**

`res.json()` → **send JSON response**

`res.status()` → **set status code**

Q19. What is the purpose of `res.redirect()`?

`res.redirect()` tells the client to **make a request to a different URL** by sending an HTTP redirect response, such as `302 Found` by default.

Role of `res.redirect()`:

1. Redirect users to another URL or route.
2. Commonly used after form submissions or authentication flows.
3. Can redirect to internal or external URLs.

Example:

```
app.get("/old-page", (req, res) => {  
  res.redirect("/new-page");  
});
```

External redirect:

```
res.redirect("https://example.com");
```

Permanent redirect:

```
res.redirect(301, "/new-page");
```

Key point:

`res.redirect()` **does not directly send the final resource**; it tells the client where to make the next request.

Q20. What is the difference between `req` and `res` objects?

In Express, `req` (**request**) contains information coming **from the client**, while `res` (**response**) provides methods to send data **back to the client**.

Feature	<code>req</code> (Request)	<code>res</code> (Response)
Represents	Incoming client request	Server response
Direction	Client → Server	Server → Client
Contains	Params, query, body, headers	Status, headers, response body
Common methods	<code>req.get()</code> , <code>req.header()</code>	<code>res.send()</code> , <code>res.json()</code>

Example	<code>req.body</code>	<code>res.json()</code>
---------	-----------------------	-------------------------

Example:

```
app.post("/users", (req, res) => {
  const name = req.body.name;

  res.status(201).json({
    message: `User ${name} created`
  });
});
```

Flow:

```
Client
  ↓ req
Express Server
  ↓ res
Client
```

Key point:

req → read/process incoming data

res → send the response back to the client.

Q21. How do you access request headers in Express?

Request headers are available through the **req object**. You can use `req.headers` to access all headers or `req.get()` / `req.header()` to retrieve a specific header.

Ways to access headers:

1. `req.headers` → Get all headers.
2. `req.get("Header-Name")` → Get a specific header.
3. `req.header("Header-Name")` → Alias of `req.get()`.

Example:

```
app.get("/users", (req, res) => {
  console.log(req.headers);
  console.log(req.get("Authorization"));

  res.send("OK");
});
```

```
});
```

For an Authorization header:

Authorization: Bearer <token>

You can access it with:

```
const token = req.get("Authorization");
```

Key point:

Use **req.get()** when you need a specific header and **req.headers** when you need the complete set of request headers.

Q22. How do you set response headers in Express?

You can set response headers using **res.set()**, **res.header()**, or **res.setHeader()**. These methods add metadata to the HTTP response sent to the client.

Ways to set headers:

1. **res.set()** → Set one or multiple headers.
2. **res.header()** → Alias for **res.set()**.
3. **res.setHeader()** → Node.js HTTP method available on the response object.

Example:

```
app.get("/", (req, res) => {  
  res.set("X-App-Version", "1.0");  
  res.json({ message: "Hello" });  
});
```

Multiple headers:

```
res.set({  
  "X-App-Version": "1.0",  
  "Cache-Control": "no-cache"  
});
```

Key point: Set response headers **before sending the response body**, because headers are generally sent when the response is committed.

Q23. How do you send a custom HTTP status code in Express?

Use `res.status()` to set the HTTP status code, then send the response using `res.send()` or `res.json()`.

Example:

```
app.get("/users", (req, res) => {  
  res.status(404).json({  
    error: "User not found"  
  });  
});
```

Common examples:

```
res.status(200).json({ message: "Success" });  
res.status(201).json({ message: "Created" });  
res.status(400).json({ error: "Bad request" });  
res.status(500).json({ error: "Server error" });
```

Key point:

`res.status(code)` sets the HTTP status, while `res.json()` or `res.send()` sends the response body.

Q24. How do you send a JSON error response from an Express API?

Use `res.status()` to set an appropriate HTTP status code and `res.json()` to send a structured JSON error response.

Example:

```
app.get("/users/:id", (req, res) => {  
  const user = null;  
  
  if (!user) {  
    return res.status(404).json({  
      success: false,  
      message: "User not found"  
    });  
  }  
});
```

Typical structure:

```
{
  "success": false,
  "message": "User not found"
}
```

Common status codes:

Cod e	Use
400	Invalid request
401	Not authenticated
403	Not authorized
404	Resource not found
500	Server error

Key point:

Use **meaningful HTTP status codes + a consistent JSON structure** so frontend clients can reliably handle API errors.

Q25. What happens if you don't send a response from an Express route?

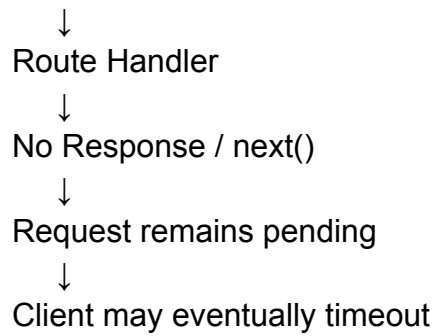
If an Express route neither **sends a response** nor **passes control using `next()`**, the request remains **open and eventually may time out**. The client keeps waiting because Express has not completed the response.

Example:

```
app.get("/users", (req, res) => {
  console.log("Request received");
  // No res.send(), res.json(), or next()
});
```

The request will stay pending:

Client Request



Correct:

```
app.get("/users", (req, res) => {  
  res.json({ users: [] });  
});
```

Or, when passing to another middleware:

```
app.use((req, res, next) => {  
  next();  
});
```

Key point:

Every request should eventually **send a response or pass control to another middleware using `next()`**.

Middleware

Middleware

Q26. What are the different types of middleware in Express?

Express middleware can be categorized based on **where and how it is used** in the request-response cycle.

Types of Middleware:

1. **Application-level middleware** → Attached to the Express app using `app.use()` or `app.METHOD()`.
2. **Router-level middleware** → Attached to an Express **Router** using `router.use()` or `router.METHOD()`.
3. **Built-in middleware** → Provided by Express, such as `express.json()` and `express.urlencoded()`.
4. **Third-party middleware** → Installed from npm, such as `cors`, `morgan`, etc.
5. **Error-handling middleware** → Handles errors using `(err, req, res, next)`.

Example:

```
// Built-in
app.use(express.json());

// Application-level
app.use(logger);

// Router-level
router.use(auth);

// Error-handling
app.use((err, req, res, next) => {
  res.status(500).json({ error: "Server error" });
});
```

Type	Example	Purpose
Application-level	<code>app.use()</code>	App-wide processing
Router-level	<code>router.use()</code>	Route-group processing

Built-in	<code>express.json()</code>	Parse request data
Third-party	<code>cors()</code>	Additional functionality
Error-handling	<code>(err, req, res, next)</code>	Handle errors

Key point:

Express middleware can handle **requests, authentication, parsing, logging, errors, and many other cross-cutting concerns.**

Q27. What is built-in middleware in Express?

Built-in middleware is middleware that comes **directly with Express**, so you don't need to install a separate npm package. It provides common functionality for processing requests.

Common Built-in Middleware:

1. `express.json()` → Parses JSON request bodies.
2. `express.urlencoded()` → Parses URL-encoded form data.
3. `express.static()` → Serves static files such as HTML, CSS, images, etc.

Example:

```
const express = require("express");
const app = express();

app.use(express.json());

app.use(express.static("public"));
```

Now:

- `express.json()` → Makes JSON data available through `req.body`.
- `express.static("public")` → Serves files from the `public` directory.

Key point:

Built-in middleware provides **common functionality without requiring additional npm packages.**

Q28. What does `express.json()` do?

`express.json()` is built-in Express middleware that **parses incoming requests with JSON payloads** and makes the parsed data available through `req.body`.

Role of `express.json()`:

1. Parses `application/json` request bodies.
2. Converts JSON data into a JavaScript object.
3. Makes the data available through `req.body`.

Example:

```
app.use(express.json());

app.post("/users", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
Request:
{
  "name": "Alex",
  "age": 22
}
Then:
req.body.name; // "Alex"
req.body.age;  // 22
```

Key point:

`express.json()` allows Express to **read and parse JSON request bodies** and expose them through `req.body`.

Q29. What does `express.urlencoded()` do?

`express.urlencoded()` is built-in Express middleware that **parses URL-encoded request bodies**, commonly sent by HTML forms using `application/x-www-form-urlencoded`.

Role of `express.urlencoded()`:

1. Parses form-encoded request data.
2. Converts it into a JavaScript object.
3. Makes the data available through `req.body`.

Example:

```
app.use(express.urlencoded({ extended: true }));

app.post("/login", (req, res) => {
  console.log(req.body);
  res.json(req.body);
});
```

Form data:
username=Alex&password=1234

Then:

```
req.body.username; // "Alex"
req.body.password; // "1234"
```

`extended` option:

Option	Behavior
<code>extended: false</code>	Uses simpler parsing
<code>extended: true</code>	Allows richer/nested objects

Key point:

`express.urlencoded()` is mainly used to **parse HTML form data** and make it available through `req.body`.

Q30. What is third-party middleware in Express? Give examples.

Third-party middleware is middleware developed outside Express and installed separately, usually through **npm**. It adds functionality that Express doesn't provide by default.

Examples:

1. **cors** → Enables/configures Cross-Origin Resource Sharing.
2. **morgan** → Logs HTTP requests.
3. **helmet** → Adds security-related HTTP headers.
4. **cookie-parser** → Parses cookies.
5. **multer** → Handles **multipart/form-data**, commonly for file uploads.

Example:

```
const cors = require("cors");
const morgan = require("morgan");

app.use(cors());
app.use(morgan("dev"));
```

Install:

```
npm install cors morgan
```

Key point:

Third-party middleware extends Express with **additional functionality** without you having to implement everything yourself.

Q31. How would you create your own custom middleware?

Custom middleware is simply a function with access to **req**, **res**, and **next**. It can perform some logic and then call **next()** to pass control to the next middleware or route.

Steps:

1. Create a function (**req**, **res**, **next**).
2. Perform your required logic.
3. Call **next()** if the request should continue.
4. Register it using **app.use()** or on a specific route.

Example:

```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

app.use(logger);
```

Route-specific middleware:

```
const auth = (req, res, next) => {
  if (!req.headers.authorization) {
    return res.status(401).json({ error: "Unauthorized" });
  }

  next();
};

app.get("/admin", auth, (req, res) => {
  res.send("Admin Dashboard");
});
```

Key point: Custom middleware is useful for reusable logic such as **logging, authentication, validation, authorization, and request processing**.

Q32. How can middleware be used for logging requests?

Middleware can inspect the incoming **request method, URL, headers, and other details**, log them, and then call `next()` to continue the request.

Role of Logging Middleware:

1. Records HTTP method and URL.
2. Helps debug and monitor API requests.
3. Can record response status and timing.
4. Helps identify slow or failed requests.

Example:

```
const logger = (req, res, next) => {  
  console.log(`${req.method} ${req.url}`);  
  next();  
};  
  
app.use(logger);
```

Request:

```
GET /users
```

Output:

```
GET /users
```

Logging response status and time:

```
const logger = (req, res, next) => {  
  const start = Date.now();  
  
  res.on("finish", () => {  
    console.log(  
      `${req.method} ${req.url} ${res.statusCode} ${Date.now() -  
start}ms`  
    );  
  });  
  
  next();  
};
```

Key point:

Logging middleware runs during every matching request and can capture **request details, response status, and performance metrics**.

Q33. How can middleware be used to authenticate users?

Authentication middleware verifies the user's **identity/credentials** before allowing the request to reach the protected route. If authentication fails, it returns an error response; otherwise, it calls `next()`.

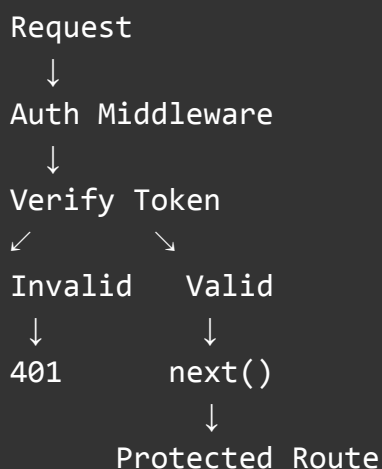
How it works:

1. Extract credentials/token from the request.
2. Verify the credentials.
3. Attach the authenticated user to `req`.
4. Call `next()` if valid.
5. Return `401 Unauthorized` if invalid.

Example:

```
const auth = (req, res, next) => {  
  const token = req.headers.authorization;  
  
  if (!token) {  
    return res.status(401).json({  
      error: "Unauthorized"  
    });  
  }  
  
  // Verify token here  
  req.user = { id: 1 };  
  next();  
};  
  
app.get("/profile", auth, (req, res) => {  
  res.json(req.user);  
});
```

Flow:



Key point: Authentication middleware acts as a **gatekeeper**, allowing only authenticated requests to access protected routes.

Q34. What happens if middleware neither sends a response nor calls `next()`?

The request will **remain pending** because Express has no instruction to continue to the next middleware/route or finish the response. Eventually, the client may **timeout**.

Example:

```
app.use((req, res, next) => {
  console.log("Middleware executed");

  // No next()
  // No res.send() / res.json()
});
```

Flow:

```
Request
↓
Middleware
↓
❌ No next()
❌ No Response
↓
Request Hangs
↓
Client Timeout
```

Correct:

```
app.use((req, res, next) => {
  console.log("Middleware executed");
  next();
});
```

Or terminate the request:

```
app.use((req, res) => {
  res.status(403).json({ error: "Forbidden" });
});
```

Key point:

Every middleware must either **call `next()`** or **complete the response** (unless it intentionally keeps the request open).

Q35. Where should error-handling middleware be placed in an Express application?

Error-handling middleware should generally be placed **after all routes and other middleware**. Express processes middleware in registration order, so the error handler needs to be registered after the code that can generate errors.

Example:

```
app.use(express.json());

app.use("/api/users", userRoutes);

// Global error handler -- last
app.use((err, req, res, next) => {
  console.error(err);

  res.status(500).json({
    error: "Internal Server Error"
  });
});
```

Flow:

```
Request
  ↓
Middleware
  ↓
Routes
  ↓
next(error)
```

```
graph TD; A[ ] --> B[Error Handler]; B --> C[Response];
```

↓
Error Handler
↓
Response

Key point:

Put the global error-handling middleware **at the end of the middleware/route chain** so it can handle errors passed from earlier handlers.

Rate Limiting

Rate Limiting

Q36. What is rate limiting in an API?

Rate limiting restricts how many requests a client can make to an API within a specific time period. It helps prevent **abuse, brute-force attacks, excessive resource usage, and denial-of-service attempts**.

Role of Rate Limiting:

1. Controls request frequency per client/IP/user.
2. Protects server resources.
3. Prevents API abuse and brute-force attacks.
4. Ensures fair usage among clients.

Example:

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  limit: 100
});

app.use("/api", limiter);
```

This allows a client to make up to **100 requests per minute** to `/api`.

When the limit is exceeded, the server typically responds with:

```
429 Too Many Requests
```

Key point:

Rate limiting = controlling how frequently clients can access an API.

Q37. Why do we need rate limiting in Express applications?

Rate limiting protects an Express API from **excessive or abusive requests** and prevents a single client from consuming too many server resources.

Why use it?

1. **Prevent abuse** → Limits automated or malicious requests.
2. **Protect resources** → Reduces excessive CPU, memory, and database usage.
3. **Prevent brute-force attacks** → Especially useful for login and OTP endpoints.
4. **Improve availability** → Helps prevent one client from overwhelming the API.
5. **Fair usage** → Prevents one client from dominating API capacity.

Example:

```
app.use("/api/login", limiter);  
If a client exceeds the configured limit:  
Too many requests  
↓  
429 Too Many Requests
```

Key point:

Rate limiting is a **protective layer** that controls request frequency and helps keep APIs stable, secure, and available.

Q38. What problems can happen if an API has no rate limiting?

Without rate limiting, a client can send **an excessive number of requests**, potentially exhausting server resources and making the API slow or unavailable.

Problems:

1. **API abuse** → Automated clients can flood endpoints with requests.
2. **Brute-force attacks** → Login/password endpoints become easier to attack.
3. **Resource exhaustion** → Excessive CPU, memory, database connections, or bandwidth usage.
4. **Denial of Service** → Large request volumes can make the API unavailable.
5. **Higher infrastructure costs** → More requests can increase compute, database, and bandwidth usage.
6. **Poor performance for other users** → One client can consume a disproportionate share of resources.

Example:

```
No Rate Limit  
↓  
100,000 requests
```

```
↓  
Server / Database overloaded  
↓  
High latency  
↓  
Requests fail
```

Key point:

Without rate limiting, an API is more vulnerable to **abuse, resource exhaustion, brute-force attacks, and availability problems**.

Q39. How does a rate limiter decide whether to allow or reject a request?

A rate limiter tracks how many requests a client makes within a defined **time window** or according to a request-rate policy. If the client is within the allowed limit, the request proceeds; otherwise, it is rejected.

1. Identify the client → IP, user ID, API key, etc.
2. Check the client's current request count/token state.
3. Compare it with the configured limit.
4. **Allow** the request if within the limit.
5. **Reject with 429** if the limit is exceeded.

Example:

```
limit = 100 requests / minute
```

Client requests:

1 → Allow

50 → Allow

100 → Allow

101 → Reject (429)

Common algorithms:

Algorithm	Idea
Fixed Window	Count requests in fixed time windows
Sliding Window	Track requests over a moving time window

Token Bucket	Requests consume tokens; tokens refill over time
Leaky Bucket	Processes requests at a controlled rate

Key point: A rate limiter identifies the client, checks its usage against a policy, and allows or rejects the request accordingly.

Q40. What is the difference between rate limiting and throttling?

Rate limiting sets a maximum number of requests a client can make within a period, while **throttling** controls the rate at which requests are processed or allowed to proceed.

Feature	Rate Limiting	Throttling
Purpose	Enforce a request limit	Control request processing rate
Focus	How many requests	How fast requests are handled
When limit exceeded	Often rejects with 429	May delay, queue, or slow requests
Example	100 requests/minute	Process max 10 requests/second
Common use	API protection	Traffic/load control

Example:

```
Rate Limiting:
100 requests/minute
Request #101 → 429 Too Many Requests

Throttling:
10 requests/second
Extra requests → delayed/queued
```

Key point:

Rate limiting = limit the amount of requests

Throttling = control the speed/rate at which requests are processed.

Q41. What is a 429 Too Many Requests status code?

429 Too Many Requests is an HTTP status code returned when a client sends more requests than the server's configured rate limit within a given period.

When is it used?

1. Client exceeds the API rate limit.
2. Protects the server from excessive traffic.
3. Commonly used for login, OTP, search, and public APIs.
4. The response may include a **Retry-After** header indicating when the client should retry.

Example:

```
if (requestCount > limit) {  
  return res  
    .status(429)  
    .set("Retry-After", "60")  
    .json({  
      error: "Too many requests"  
    });  
}
```

Flow:

```
Client  
↓  
101st request  
↓  
Rate Limiter  
↓  
429 Too Many Requests
```

Key point:

429 means the client has exceeded the allowed request rate and should slow down or retry later.

Q42. How would you implement rate limiting in Express?

A common approach is to use the **express-rate-limit** middleware. You configure a time window and maximum number of requests, then apply the limiter globally or to specific routes.

Steps:

1. Install **express-rate-limit**.
2. Configure the time window and request limit.
3. Apply it globally or to specific endpoints.
4. Return **429** when the limit is exceeded.

Example:

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  limit: 100
});

app.use("/api", limiter);
```

Now a client exceeding **100 requests/minute** receives:

```
429 Too Many Requests
```

For sensitive routes:

```
app.use("/api/login", limiter);
```

Key point:

Use rate limiting **globally for broad protection** and stricter limits on sensitive endpoints such as login, OTP, password reset, and expensive API operations.

Q43. What is the purpose of the `express-rate-limit` package?

`express-rate-limit` is Express middleware used to **limit the number of requests a client can make within a specified time window**. It helps protect APIs from abuse, brute-force attempts, and excessive traffic.

Role of `express-rate-limit`:

1. Tracks requests from clients.
2. Enforces a configured request limit.
3. Returns **429 Too Many Requests** when the limit is exceeded.
4. Can be applied globally or to specific routes.

Example:

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
  windowMs: 60 * 1000,
  limit: 100
});

app.use("/api", limiter);
```

This allows **100 requests per minute** within the configured limiter scope.

Key point:

`express-rate-limit` provides a simple way to add **request-rate protection to Express APIs**.

Q44. How can you configure a rate limiter for a specific route?

Create a rate limiter using `express-rate-limit` and pass it as **route-level middleware**. This ensures only that route is rate-limited.

Example:

```
const rateLimit = require("express-rate-limit");

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  limit: 5
});
```

```
});

app.post("/login", loginLimiter, (req, res) => {
  res.json({ message: "Login successful" });
});
```

Here, the `/login` route allows **5 requests per 15 minutes** per limiter key before returning `429`.

Flow:

```
POST /login
  ↓
loginLimiter
  ↓
Within limit? — No → 429
  ↓ Yes
Route Handler
  ↓
Response
```

Key point:

Pass the limiter **between the route path and route handler** to apply rate limiting only to that endpoint.

Q45. What is the difference between applying rate limiting globally and to a specific route?

Global rate limiting applies a rate limit to all matching requests in the application, while **route-level rate limiting** applies limits only to selected endpoints.

Feature	Global Rate Limiting	Route-Level Rate Limiting
Scope	Broad/all API routes	Specific route(s)
Configuration	One general policy	Custom per endpoint
Use case	Overall API protection	Sensitive/expensive endpoints
Example	<code>app.use(limiter)</code>	<code>app.post("/login", limiter, ...)</code>

Flexibility	Lower	Higher
-------------	-------	--------

Global:

```
app.use("/api", limiter);
```

All `/api/*` routes are protected.

Specific route:

```
app.post("/login", loginLimiter, login);
```

Only `/login` uses `loginLimiter`.

Key point:

Global → broad protection with a common policy

Route-level → targeted protection with endpoint-specific limits.

Q46. What is a window in rate limiting? For example, **100 requests per 15 minutes**.

A **window** is the period during which a rate limiter counts requests against a configured limit. For example, **100 requests per 15 minutes** means a client can make up to 100 allowed requests within that configured 15-minute window.

Example:

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  limit: 100
});
```

How it works:

15-minute window

Request 1 ... Request 100	→ Allowed
Request 101	→ 429

Important: The exact reset behavior depends on the **rate-limiting algorithm/store** being used; a fixed-window limiter resets at the end of each fixed window.

Key point: Window = time used to measure and enforce the request limit.

Q47. What happens when a user exceeds the rate limit?

When a client exceeds the configured request limit, the **rate limiter rejects further requests** according to its policy. In Express APIs, this commonly results in a **429 Too Many Requests** response.

What happens?

1. Rate limiter detects the limit has been exceeded.
2. Requests are rejected or delayed, depending on the limiter.
3. Server typically returns **HTTP 429**.
4. A **Retry-After** header may indicate when the client should retry.

Example:

```
return res.status(429).json({  
  error: "Too many requests"  
});
```

Flow:

```
Request  
↓  
Rate Limiter  
↓  
Limit exceeded?  
↓ Yes  
429 Too Many Requests
```

Key point:

Exceeding the limit usually means **temporary rejection of further requests until the client is allowed to make requests again**.

Q48. What should an API return when a request is rejected because of rate limiting?

A rate-limited API should return **HTTP 429 Too Many Requests** along with a clear error response. It can also include **Retry-After** to tell the client when it should retry.

Recommended response:

```
{
  "success": false,
  "message": "Too many requests. Please try again later."
}
```

Example:

```
res
  .status(429)
  .set("Retry-After", "60")
  .json({
    success: false,
    message: "Too many requests"
  });
```

Response should contain:

1. **Status code** → **429 Too Many Requests**
2. **Clear error message** → Explain that the limit was exceeded.
3. **Retry-After header** → Recommended retry delay, when applicable.
4. Optionally, rate-limit metadata such as remaining requests or reset information.

Key point:

A good rate-limit response should clearly communicate **"you exceeded the allowed request rate"** and, when possible, **"when you can retry."**

Q49. Why might rate limiting based only on IP address have limitations?

IP-based rate limiting is simple, but an IP address does **not always uniquely identify a user or client**. Many users can share an IP, while a single user can also appear from different IPs.

Limitations:

1. **Shared IPs** → Multiple users behind NAT/proxies may share one IP.
2. **Dynamic IPs** → A user's IP can change over time.
3. **Mobile networks** → Many users may share carrier IP addresses.
4. **VPNs/Proxies** → Users can appear from different IPs.
5. **IPv6/privacy mechanisms** → Client addressing can change or be less stable.
6. **Distributed attacks** → An attacker can use many IP addresses to bypass limits.

Example:

```
Office Network
  ↓
One Public IP
  ↓
100 Employees
  ↓
IP-based limit applies to everyone
```

If the limit is **100 requests/IP**, one employee's heavy usage could consume the quota for the entire office.

Better approach:

Use a rate-limit key appropriate to the application, such as:

```
Authenticated user ID
      +
API key
      +
IP address (when useful)
```

Key point:

IP ≠ User. For authenticated APIs, rate limiting by **user ID/API key**, often combined with IP-based protection, can provide more accurate control.

Q50. Where would you apply stricter rate limits — login APIs or normal GET APIs, and why?

I would generally apply **stricter rate limits to login APIs** because they are high-risk endpoints that can be targeted by **credential-stuffing and brute-force attacks**. Normal GET APIs can usually have higher limits depending on their cost and sensitivity.

Feature	Login API	Normal GET API
Rate limit	Stricter	Usually higher
Main concern	Brute-force / credential stuffing	Excessive usage / resource consumption
Example	5–10 attempts / 15 min	100–1000 requests / min
Response	429 after limit	429 after limit

Example:

```
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 5
});

app.post("/login", loginLimiter, login);
For a normal API:
app.get("/products", apiLimiter, getProducts);
```

Key point:

Apply **stricter limits to authentication endpoints**, but always tune limits based on the endpoint's **security risk, cost, and expected traffic**.

REST APIs

REST APIs

Check sub-tabs for complete QnA

REST API Fundamentals

REST API Fundamentals

Q1. What is a REST API?

A **REST API (Representational State Transfer API)** is an API architecture that allows clients and servers to communicate over **HTTP** using resources such as users, products, or orders. It typically uses HTTP methods like **GET**, **POST**, **PUT**, **PATCH**, and **DELETE**.

Key Characteristics:

1. **Resource-based** → Resources are identified using URLs, e.g. `/users/123`.
2. **HTTP methods** → Define operations on resources.
3. **Stateless** → Each request contains the information needed to process it.
4. **Representations** → Data is commonly exchanged as JSON.
5. **Client-server separation** → Frontend and backend are independent.

Example:

```
GET /api/users/123
Response:
{
  "id": 123,
  "name": "Alex"
}
```

Key point:

REST API = resource-based HTTP API designed around REST principles, commonly using JSON for data exchange.

Q2. What makes an API RESTful?

An API is considered **RESTful** when it follows the core principles and constraints of **REST (Representational State Transfer)**. The API should model resources and use HTTP semantics consistently.

Key REST Constraints:

1. **Client-Server** → Client and server have separate responsibilities.
2. **Stateless** → Each request contains all information needed to process it.
3. **Uniform Interface** → Consistent resource URLs and HTTP methods.

4. **Cacheable** → Responses can indicate whether they may be cached.
5. **Layered System** → Client doesn't need to know whether it is communicating directly with the server or through intermediaries.
6. **Code on Demand (optional)** → Server may send executable code to the client.

Example:

```
GET    /users      → Get users
POST   /users      → Create user
GET    /users/10   → Get user 10
PATCH /users/10   → Update user 10
DELETE /users/10   → Delete user 10
```

Key point:

A truly RESTful API follows REST's architectural constraints rather than simply being an API that uses HTTP and JSON.

Q3. What is an endpoint?

An **endpoint** is a specific **URL + HTTP method** through which a client interacts with a particular API resource or operation.

Role of an Endpoint:

1. Defines **where** the API request is sent.
2. Uses an HTTP method to define **what action** is performed.
3. Connects the request to specific backend logic.

Example:

```
GET /api/users/123
```

Here:

```
/api/users/123 → URL/path
GET → HTTP method
Together → API endpoint
```

More examples:

```
GET    /api/users
```

```
POST    /api/users
GET      /api/users/123
DELETE  /api/users/123
```

Key point:

Endpoint = HTTP method + URL/path that a client uses to access an API operation.

Q4. How would you design CRUD APIs for a resource such as **users** or **products**?

I would design the API around the **resource**, using standard HTTP methods and predictable URLs. For a **users** resource, the API would support Create, Read, Update, and Delete operations.

Operation	Method	Endpoint	Purpose
Create	POST	/users	Create a user
Read all	GET	/users	Get users
Read one	GET	/users/:id	Get a specific user
Update	PATCH	/users/:id	Partially update a user
Delete	DELETE	/users/:id	Delete a user

Express Example:

```
router.post("/users", createUser);
router.get("/users", getUsers);
router.get("/users/:id", getUser);
router.patch("/users/:id", updateUser);
router.delete("/users/:id", deleteUser);
```

Example request:

```
PATCH /users/123
Content-Type: application/json
{ "name": "Neeraj" }
```

Design principles:

1. Use **nouns**, not verbs → `/users`, not `/getUsers`.
2. Use HTTP methods to define the operation.
3. Return appropriate **HTTP status codes**.
4. Use consistent JSON response/error structures.
5. Validate input and authenticate/authorize protected operations.

Key point:

Resource-based URLs + correct HTTP methods + proper status codes = clean CRUD API design.

Q5. What is the difference between request headers, body, query parameters, and URL parameters?

They are different parts of an HTTP request used to send **metadata, data, or resource information** from the client to the server.

Part	Purpose	Example	Express Access
Headers	Metadata/information about request	<code>Authorization: Bearer token</code>	<code>req.headers</code>
Body	Data sent to the server	<code>{ "name": "Neeraj" }</code>	<code>req.body</code>
Query Params	Filtering, sorting, pagination	<code>/users?page=2&limit=10</code>	<code>req.query</code>
URL Params	Identify a specific resource	<code>/users/123</code>	<code>req.params</code>

Example Request:

```
POST /users/123?role=admin
Authorization: Bearer token123
Content-Type: application/json

{
  "name": "Neeraj"
}
```

Here:

```
Headers      → Authorization, Content-Type
URL Parameter → 123
Query        → role=admin
Body         → { "name": "Neeraj" }
```

Key point:

Headers → metadata | **Body** → payload | **Query params** → filtering/options | **URL params** → resource identification.

Q6. What is JSON, and why is it commonly used in REST APIs?

JSON (JavaScript Object Notation) is a lightweight, text-based data format used to represent structured data. REST APIs commonly use JSON because it is **simple, human-readable, compact, and widely supported** across programming languages.

Why JSON is used:

1. **Easy to read and write** → Simple key-value structure.
2. **Language-independent** → Supported by Java, Python, JavaScript, Go, etc.
3. **Lightweight** → Smaller and easier to transfer over HTTP than many verbose formats.
4. **Easy to parse** → Most languages provide built-in or standard JSON support.

Example:

```
{
  "id": 101,
  "name": "Neeraj",
  "role": "Developer"
}
```

REST API response:

```
Content-Type: application/json
```

Key point:

JSON provides a simple, interoperable format for exchanging structured data between clients and **REST APIs**.

Q7. What is Postman, and how do you use it to test an API?

Postman is an API development and testing tool used to **send HTTP requests**, **inspect responses**, **test endpoints**, and **automate API tests**.

How to use Postman:

1. Enter the API **URL** and select the HTTP method.
2. Add **headers**, query parameters, or request body if required.
3. Send the request.
4. Check the **status code**, **response body**, **headers**, and **response time**.
5. Create collections/tests to repeatedly test multiple endpoints.

Example:

```
POST http://localhost:3000/api/users
Content-Type: application/json
Body:
{
  "name": "Neeraj",
  "email": "neeraj@example.com"
}
Expected response:
{ "id": 101, "name": "Neeraj" }
```

What you can test:

Test	Example
Method	GET, POST, PATCH, DELETE
Status	200, 201, 400, 404, 500
Headers	Authorization, Content-Type
Body	JSON payload
Authentication	JWT/API keys
Performance	Response time

Key point: **Postman** lets you test and debug APIs without needing to build the frontend client first.

Q8. What is the difference between HTTP request and HTTP response?

An **HTTP request** is sent by the **client to the server** to ask for an operation or resource. An **HTTP response** is sent by the **server back to the client** with the result.

Feature	HTTP Request	HTTP Response
Direction	Client → Server	Server → Client
Contains	Method, URL, headers, optional body	Status code, headers, optional body
Example method	GET, POST	200, 404, 500
Purpose	Request an action/resource	Provide the result

Example:

Request:

```
GET /api/users/123
Authorization: Bearer token
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 123,
  "name": "Neeraj"
}
```

Key point:

Request = client asks for something

Response = server tells the client what happened.

Q9. What are HTTP headers? Give examples of commonly used headers.

HTTP headers are key-value pairs sent with an HTTP request or response that provide **metadata and additional instructions** about the message.

Role of Headers:

1. Provide information about the request/response.
2. Control caching, authentication, content type, etc.
3. Help the client and server understand how to process the message.

Common Request Headers:

Header	Purpose	Example
Authorization	Authentication credentials	Bearer <token>
Content-Type	Format of request body	application/json
Accept	Response formats client accepts	application/json
User-Agent	Client information	Mozilla/5.0

Common Response Headers:

Header	Purpose	Example
Content-Type	Format of response body	application/json
Cache-Control	Caching behavior	no-cache
Set-Cookie	Set a cookie	session=abc
Location	Redirect/resource location	/users/123

Example:

```
POST /api/users  
Content-Type: application/json  
Authorization: Bearer token123
```

Key point:

HTTP headers carry metadata and control how requests and responses should be interpreted or handled.

Q10. What is the difference between PUT and PATCH?

Q11. How should an API respond when a requested resource does not exist?

Q12. How should you validate data received through an API?

HTTP Methods & Status Codes

HTTP Methods & Status Codes

What is the difference between POST and PUT?

What is the difference between POST and PATCH?

What is the difference between DELETE and GET from an API perspective?

What does idempotent mean in REST APIs?

Which HTTP methods are generally idempotent, and what does that mean practically?

What is the difference between 200 OK and 201 Created?

When should an API return 204 No Content?

What is the difference between 400 Bad Request and 422 Unprocessable Content?

What is the difference between 401 Unauthorized and 403 Forbidden?

When should an API return 404 Not Found?

When should an API return 409 Conflict?

When should an API return 429 Too Many Requests?

What does a 500 Internal Server Error mean?

What is the difference between client errors (4xx) and server errors (5xx)?

API Requests & Responses

API Requests & Responses

What is the purpose of the Accept header?

What is the purpose of the Content-Type header?

What does **Content-Type: application/json** mean?

What is the difference between path parameters and query parameters?

When should you use query parameters for filtering, searching, and sorting?

How would you implement pagination in a REST API?

What are **page** and **limit** parameters?

How would you implement sorting and filtering in an API?

What should a good API response structure look like?

How should an API return multiple validation errors?

How should an API return error messages consistently?

What is the purpose of an API's response status code versus response body?

API Design

API Design

How would you design a REST API for a blog application?

How would you design REST APIs for users, posts, and comments?

How would you design an API for login and registration?

How would you protect an API endpoint so that only authenticated users can access it?

How would you restrict an API endpoint based on a user's role?

What is API versioning, and why might it be needed?

What is the difference between `/api/v1/users` and `/api/v2/users`?

What is pagination, and why shouldn't an API return thousands of records at once?

What is filtering vs searching vs sorting in an API?

What is API documentation, and why is it useful?

What is Swagger/OpenAPI, at a beginner level?

What makes an API easy for frontend developers to consume?

MongoDB Integration

MongoDB Integration

Check sub-tabs for complete QnA

MongoDB Fundamentals

MongoDB Fundamentals

What is MongoDB?

What is a MongoDB document?

What is a MongoDB collection?

What is a MongoDB database?

What is the difference between a document and a collection?

Why is MongoDB commonly used with Node.js?

What are documents and collections?

SQL vs MongoDB

How do you connect Node/Express to MongoDB?

What is Mongoose?

What are Mongoose Schema and Model?

Basic CRUD with Mongoose

What is BSON, and how is it different from JSON?

What is an `_id` field in MongoDB?

What is an ObjectId?

Why does MongoDB automatically generate an ObjectId?

Can MongoDB documents have different fields within the same collection?

What does schema flexibility mean in MongoDB?

What is the difference between embedded documents and references?

When would you embed data inside a document?

When would you use references between documents?

What are arrays in MongoDB documents?

Can a MongoDB document contain nested objects?

CRUD & Queries

CRUD & Queries

How do you insert a document into MongoDB?

What is the difference between `insertOne()` and `insertMany()`?

How do you find documents in MongoDB?

What is the difference between `find()` and `findOne()`?

How do you find a document using its `_id`?

How do you update a document in MongoDB?

What is the difference between `updateOne()` and `updateMany()`?

What is the purpose of the `$set` operator?

How do you delete a document?

What is the difference between `deleteOne()` and `deleteMany()`?

What happens if an update query doesn't find a matching document?

What are MongoDB query operators?

What are `$gt`, `$gte`, `$lt`, and `$lte` used for?

What are `$eq` and `$ne`?

What are `$in` and `$nin`?

What is the `$or` operator?

What is the `$and` operator?

How do you query documents based on a nested field?

How do you query documents where an array contains a particular value?

How do you select only specific fields from a MongoDB document?

Mongoose

Mongoose

Why do we use Mongoose instead of directly using the MongoDB driver?

What is a Mongoose Schema?

What is a Mongoose Model?

What is the difference between a Schema and Model?

How do you define field types in a Mongoose schema?

How do you make a Mongoose field required?

What are default values in Mongoose?

What are Mongoose validation rules?

What is the difference between MongoDB validation and Mongoose validation?

What is the purpose of the **unique** option in a Mongoose schema?

Does **unique: true** perform normal Mongoose validation?

How do you define a reference to another MongoDB document in Mongoose?

What is **ObjectId** in a Mongoose schema?

What is **populate()** in Mongoose?

When would you use **populate()**?

Practical Questions

Practical Questions

How do you fetch MongoDB data and return it through an Express API?

How do you create a POST API that saves data to MongoDB?

How do you update a MongoDB document through an Express API?

How do you delete a MongoDB document through an Express API?

How do you handle MongoDB errors in an Express application?

How do you handle the case where a requested MongoDB document doesn't exist?

How do you validate incoming API data before saving it to MongoDB?

How would you prevent duplicate users from being created?

How do you store passwords in a MongoDB user collection?

How would you design a basic MongoDB schema for a User?

How would you design a basic MongoDB schema for a Product?

How would you design relationships between Users and Orders?

How would you design relationships between Users and Posts?

How would you implement search using MongoDB queries?

How would you implement pagination with MongoDB?

Indexing & Performance

Indexing & Performance

What is a MongoDB index?

Why do we need indexes?

What is the difference between a collection scan and an index scan?

How do you create an index in MongoDB?

What is a unique index?

What are the disadvantages of having too many indexes?

When should you create an index on a field?

How would you decide whether a field needs an index?

Aggregation

Aggregation

What is the MongoDB aggregation pipeline?

Why would you use aggregation instead of a normal `find()` query?

What is the `$match` stage?

What is the `$group` stage?

What is the `$sort` stage?

What is the `$project` stage?

What is the difference between `find()` and aggregation?